

Data Communications

Lecture #3

What we have discussed

- What does the Internet look like?
- What happens on the Internet

Today...

- Principles of network applications
- Web and HTTP

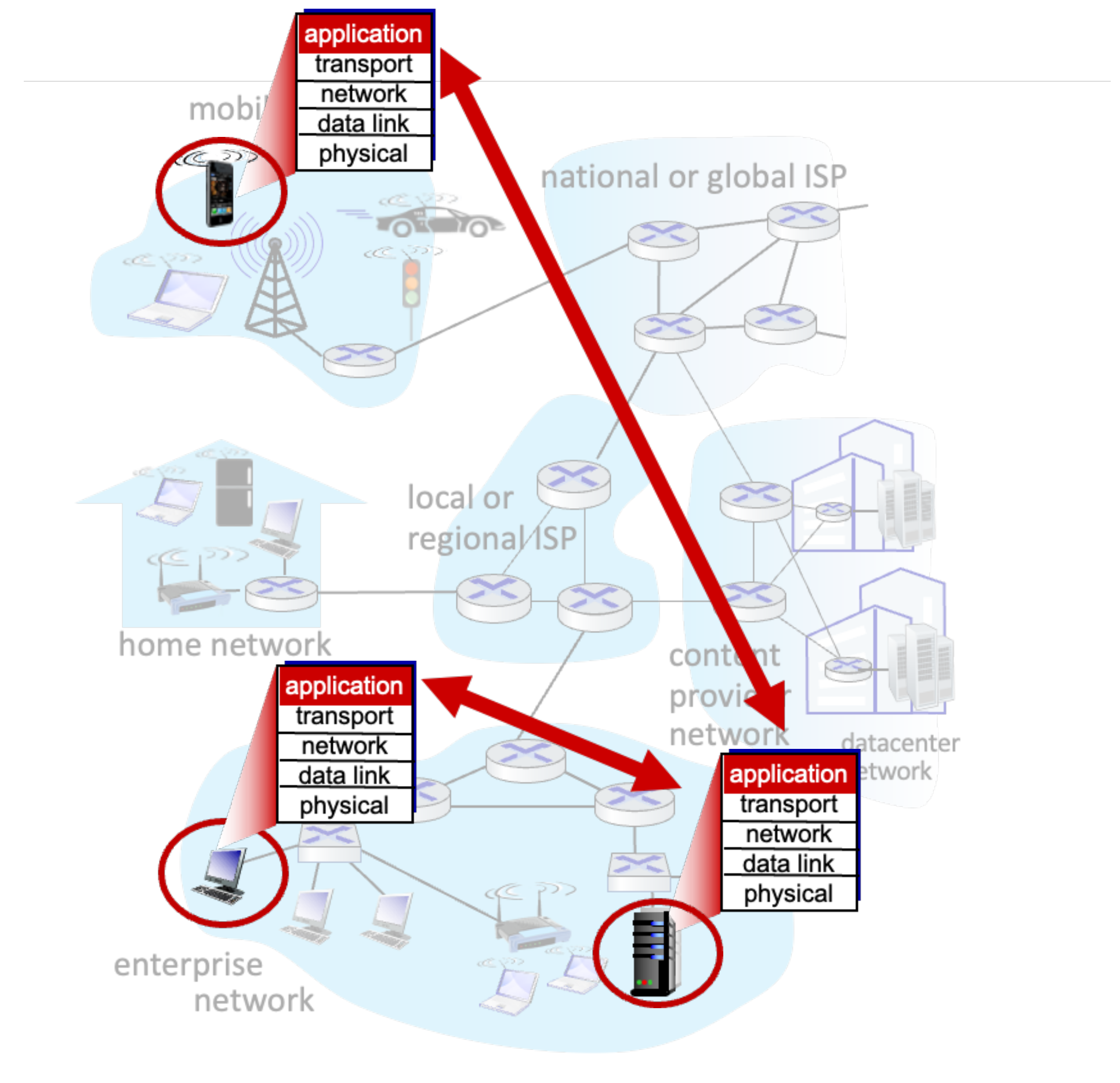
Principles of network applications

Some network apps

- Social networking
- Web
- Text messaging
- E-mail
- Multi-user network games
- Streaming stored video (Youtube, Hulu, Netflix)
- P2P file sharing
- Voice over IP (e.g., Skype)
- Real-time video conferencing (e.g., Zoom)
- Internet search
- Remote login
- ... much more apps you are using on your devices

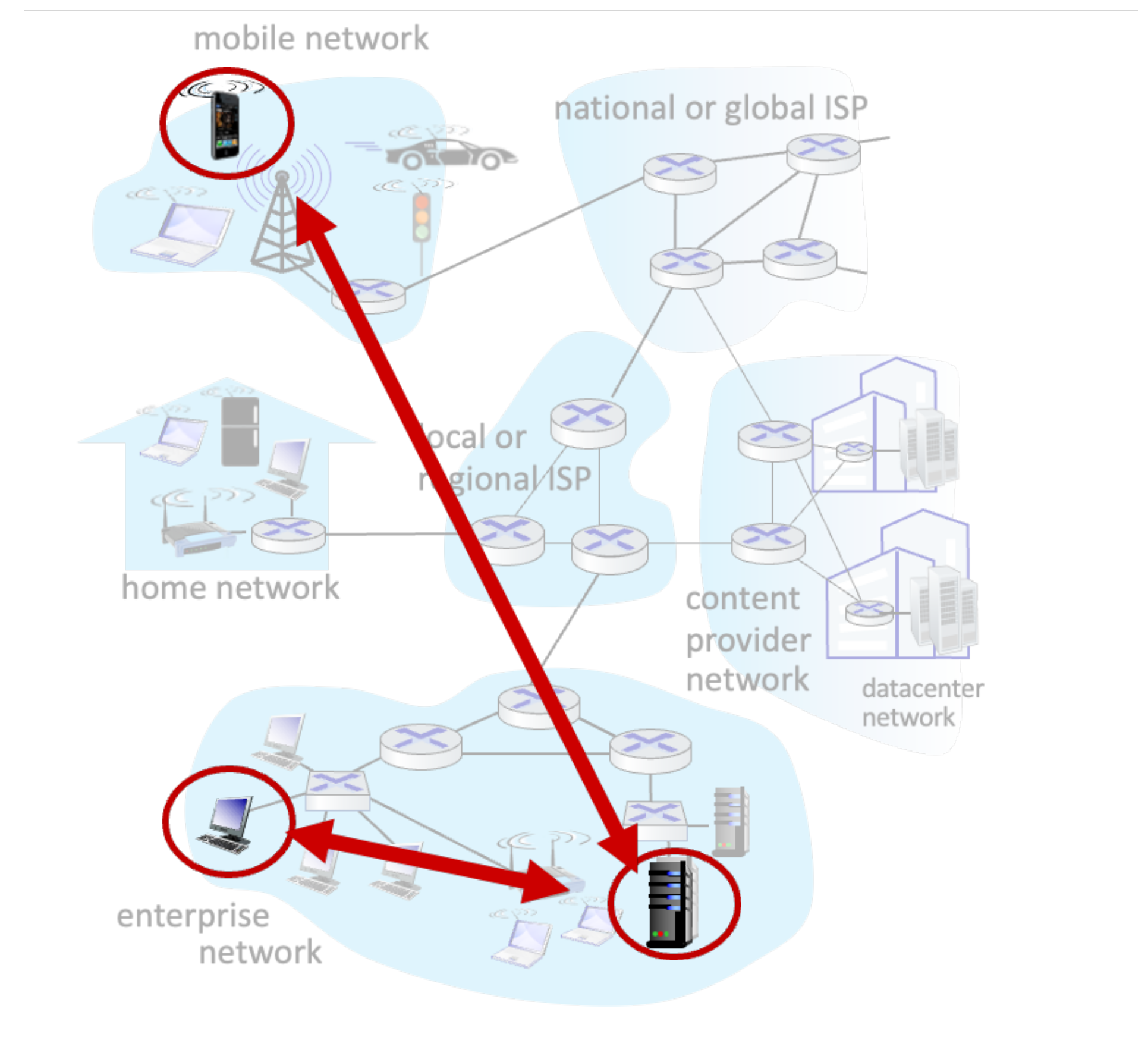
Creating a network app

- Write programs that
 - Run on (different) end systems
 - Communicate over network
 - e.g., web server software communicates with browser software
- No need to write software for network-core devices
 - Network-core devices (e.g., routers) do not run user applications
 - Applications on end systems allow for rapid app development, propagation



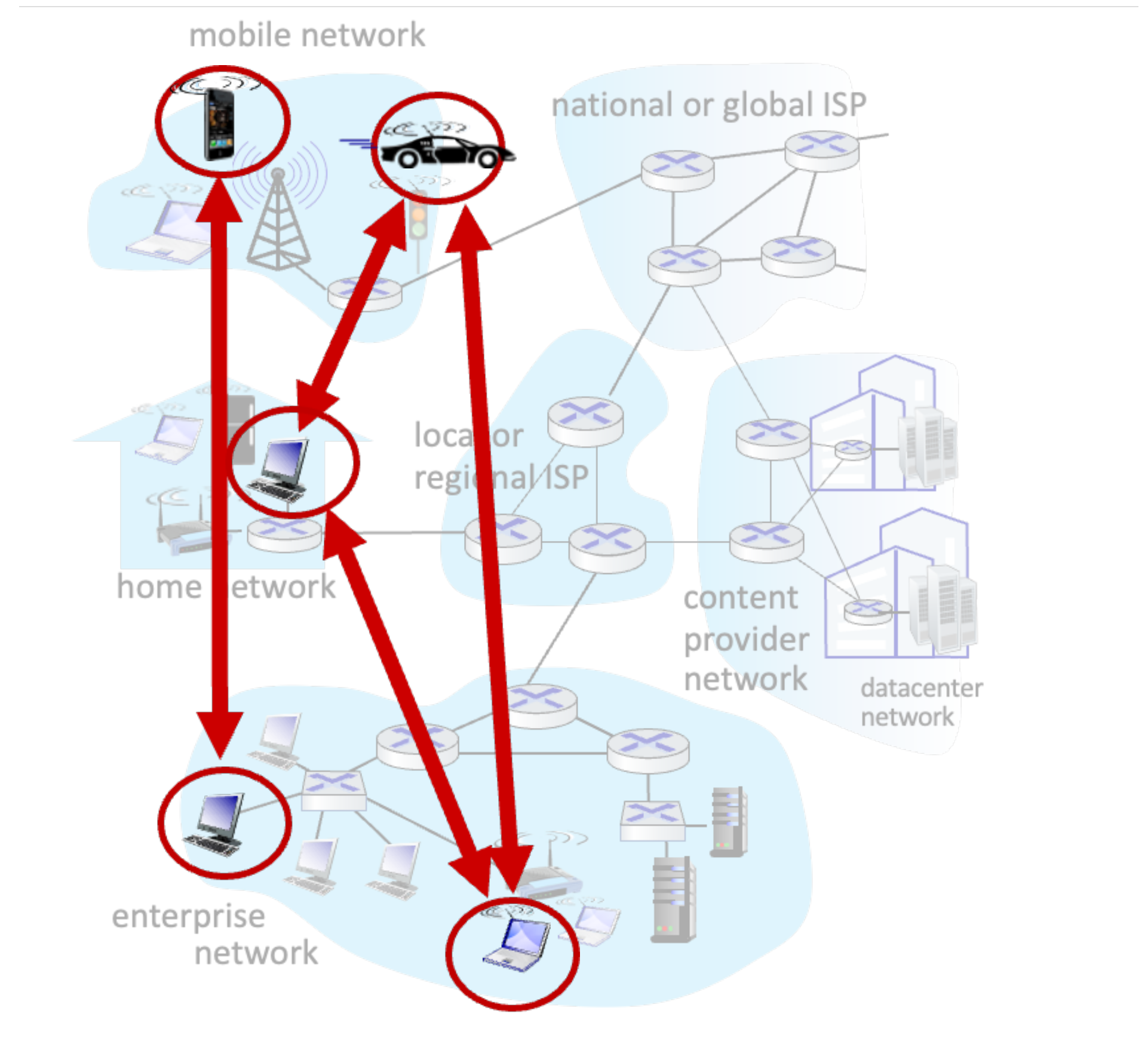
Client-Server paradigm

- Server
 - Always-on host with permanent IP address
 - Often in data centers, for scaling
- Clients
 - Contact, communicate with server
 - May be intermittently connected
 - May have dynamic IP addresses
 - Do not communicate directly with each other
 - e.g., HTTP, IMAP, FTP



Peer-to-peer architecture

- No always-on server
- Arbitrary end systems directly communicate
- Peers request service from other peers, provide service in return to other peers
 - **Self-scalability** - new peers bring new service capacity, as well as new service demands
- Peers are intermittently connected and change IP addresses
 - Complex management
- e.g., P2P file sharing (bitTorrent)

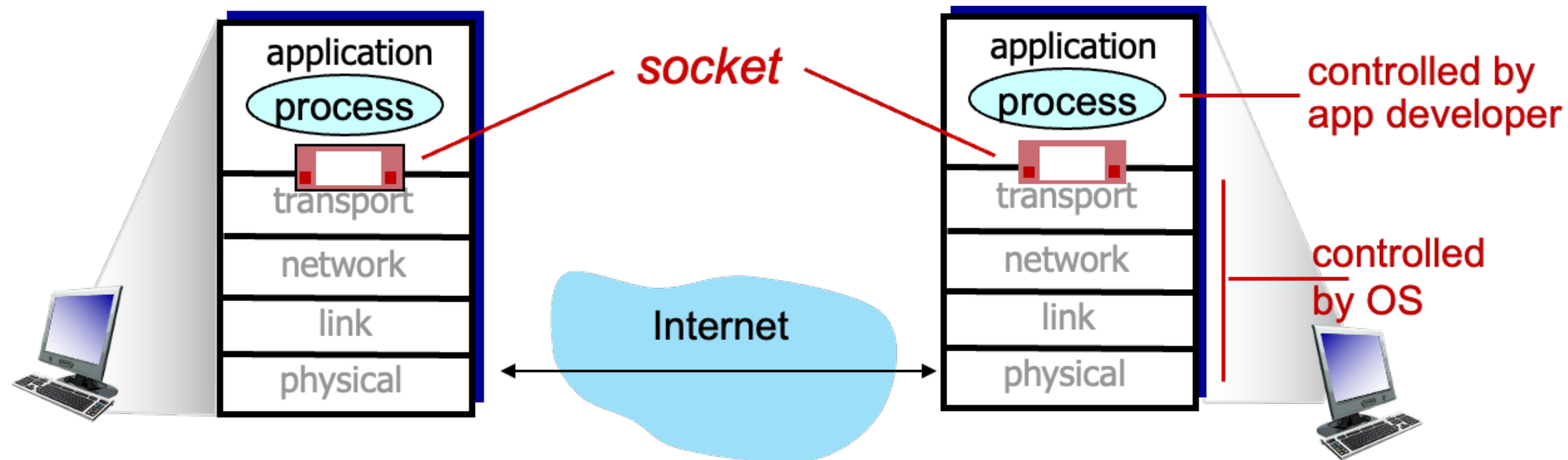


Processes communicating

- **Process** : a program running within a host
- Process communication
 - Within the same host, two processes communicate using **inter-process communication** (defined by OS)
 - Processes in different hosts communicate by exchanging messages
- Clients / Servers
 - Client process: process that initiates communication
 - Server process: process that waits to be contacted
- Applications with P2P architecture have client processes & server processes

Sockets

- Process sends/receives messages to/from its socket
- Socket analogous to door
 - Sending process shoves message out door
 - Sending process relies on transport infrastructure on the other side of door to deliver message to socket at receiving process
- Two sockets involved: one on each side



Addressing processes

- To receive messages, process must have an **identifier**
- Host device has unique 32-bit IP address
 - Q) Does IP address of a host on which process runs suffice for identifying the communicating process?

⇒ No, many processes can be running on the same host
- The **identifier** includes both **IP address** and **port number** associated with a process on host
 - e.g., HTTP server: 80, mail server: 25
- To send HTTP message to `www.icnlab.dev` web server
 - IP address: 210.115.35.29
 - Port number: 80

Application-layer protocol

- An application-layer protocol defines
 - Type of messages exchanged : e.g., request, response
 - Message syntax : what fields in messages & how fields are delineated
 - Message semantics : meaning of information in fields
 - Rules for when and how processes send & respond to messages
- Open protocols
 - Defined in RFCs, everyone has access to protocol definition
 - Allows for interoperability
 - e.g., HTTP, SMTP
 - c.f., Proprietary protocols - e.g., Skype, Zoom

What transport service does an app need?

- Data integrity

- Some apps (e.g., file transfer, web transaction) require 100% reliable data transfer
- Other apps (e.g., audio) can tolerate some loss

- Timing

- Some apps (e.g., Internet telephony, interactive games) require low delay to be “effective”

- Throughput

- Some apps (e.g., multimedia) require minimum amount of throughput to be “effective”
- Other apps (“elastic apps”) make use of whatever throughput they get

- Security

- Encryption, data integrity,

Transport service requirement

- Common apps

application	data loss	throughput	time sensitive?
file transfer/download	no loss	elastic	no
e-mail	no loss	elastic	no
Web documents	no loss	elastic	no
real-time audio/video	loss-tolerant	audio: 5Kbps-1Mbps video:10Kbps-5Mbps	yes, 10's msec
streaming audio/video	loss-tolerant	same as above	yes, few secs
interactive games	loss-tolerant	Kbps+	yes, 10's msec
text messaging	no loss	elastic	yes and no

Internet transport protocol services

- TCP service

- **Reliable transport** between sending and receiving process
- **Flow control**: sender won't overwhelm receiver
- **Congestion control**: throttle sender when network overloaded
- **Connection-oriented**: setup required between client and server processes
- **Does not provide** timing, minimum throughput guarantee, security

- UDP service

- **unreliable data transfer** between sending and receiving process
- **Does not provide** reliability, flow control, timing, throughput guarantee, security or connection setup

Then, why is there a UDP?

Internet applications, and transport protocols

application	application layer protocol	transport protocol
file transfer/download	FTP [RFC 959]	TCP
e-mail	SMTP [RFC 5321]	TCP
Web documents	HTTP [RFC 7230, 9110]	TCP
Internet telephony	SIP [RFC 3261], RTP [RFC 3550], or proprietary	TCP or UDP
streaming audio/video	HTTP [RFC 7230], DASH	TCP
interactive games	WOW, FPS (proprietary)	UDP or TCP

Securing TCP

- Vanilla TCP & UDP sockets
 - No encryption
 - Cleartext passwords sent into socket traverse Internet in cleartext (!)
- Transport Layer Security (TLS)
 - Provides encrypted TCP connections
 - Data integrity
 - End-point authentication
- TLS is implemented in application layer
 - Apps use TLS libraries, that use TCP in turn
 - Cleartext sent into “socket” traverse Internet encrypted

Web and HTTP

Web and HTTP

- Web page consists of **objects**, each of which can be stored on different web servers
- Object can be HTML file, JPEG image, Java applet, audio file, ...

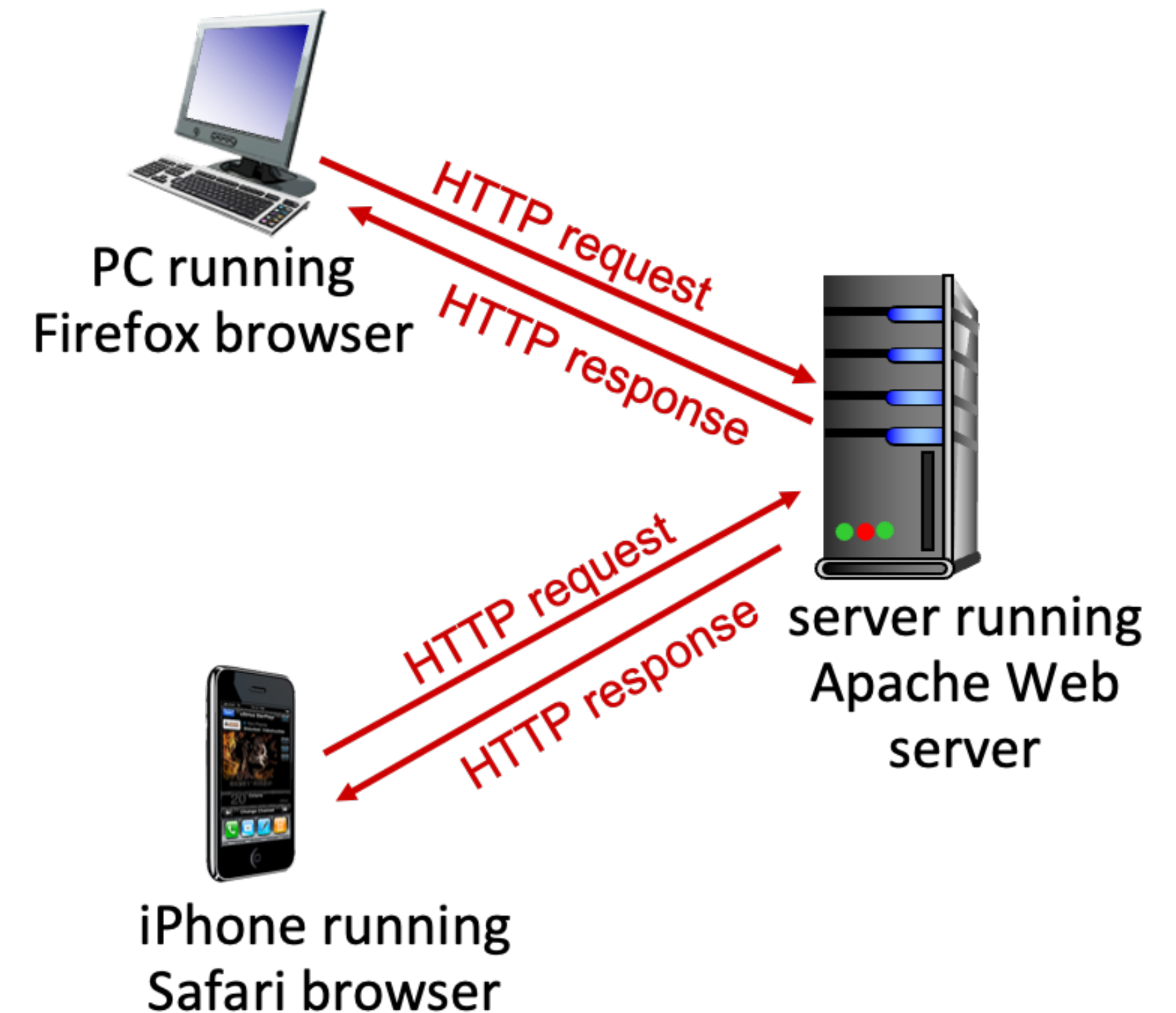
⇒ Web page consists of **base HTML-file**, which includes several **reference objects**, **each** addressable by a **URL**, e.g.,

`www.kangwon.ac.kr/cseDept/pic.gif`

host name path name

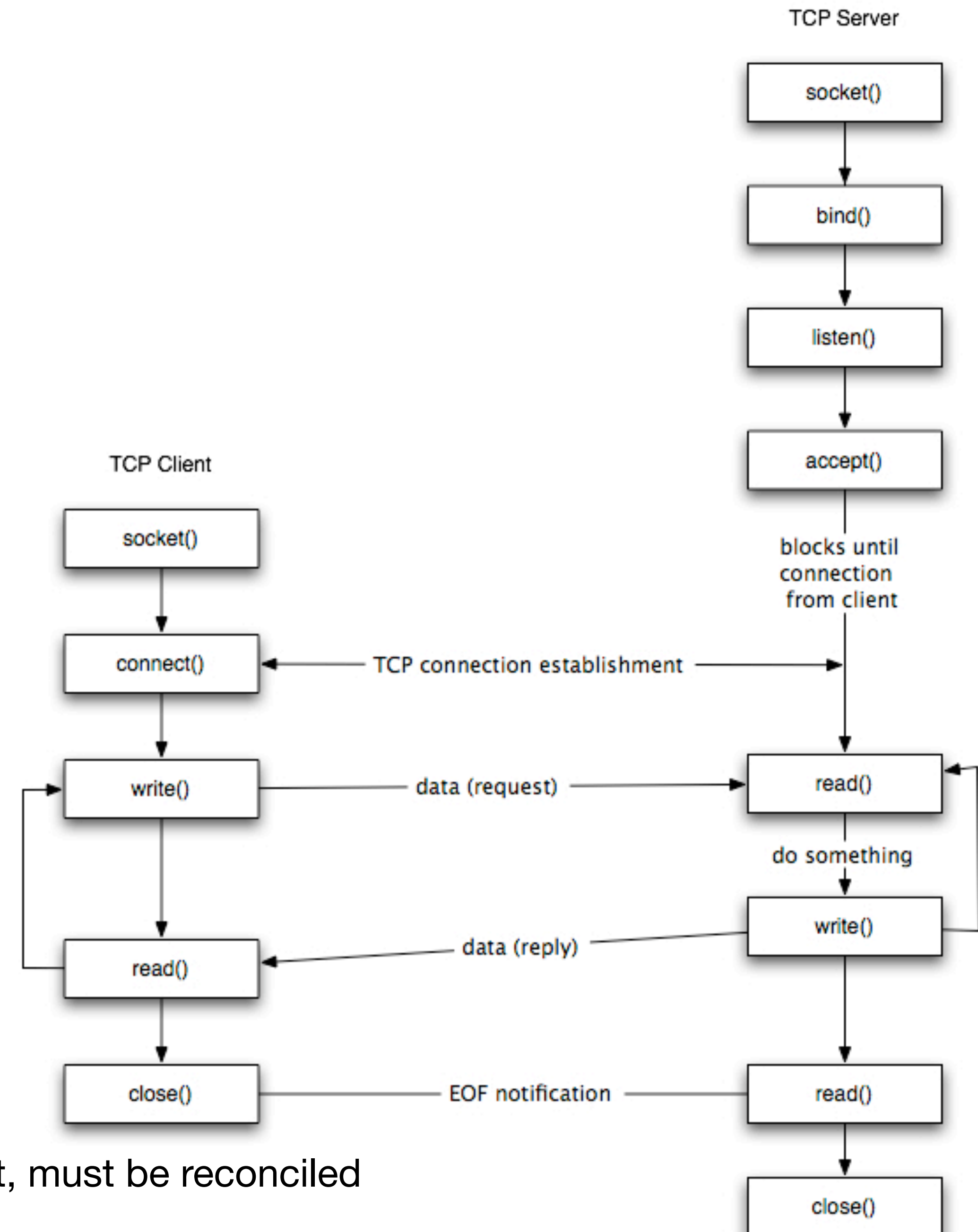
HTTP overview

- Hypertext transfer protocol (HTTP)
 - Web's application layer protocol
 - Client/server model
 - Client: browser that requests, receives, (using HTTP protocol) and display web objects
 - Server: web server sends (using HTTP protocol) objects in response to requests



HTTP overview

- HTTP uses TCP
 - Client initiates TCP connection (creates socket) to server, port 80
 - Server accepts TCP connection from client
 - HTTP messages (application-layer protocol messages) are exchanged between browser (HTTP client) and Web server (HTTP server)
 - TCP connection closed
- HTTP is “stateless”
 - Server maintains no information about past client requests
 - Protocols that maintain “state” are complex!
 - Past history (state) must be maintained
 - If server/client clashes, their views of “state” may be inconsistent, must be reconciled



HTTP connection type

- **Non-persistent HTTP**

- TCP connection opened
- **At most one object** is sent over TCP connection
- TCP connection is closed

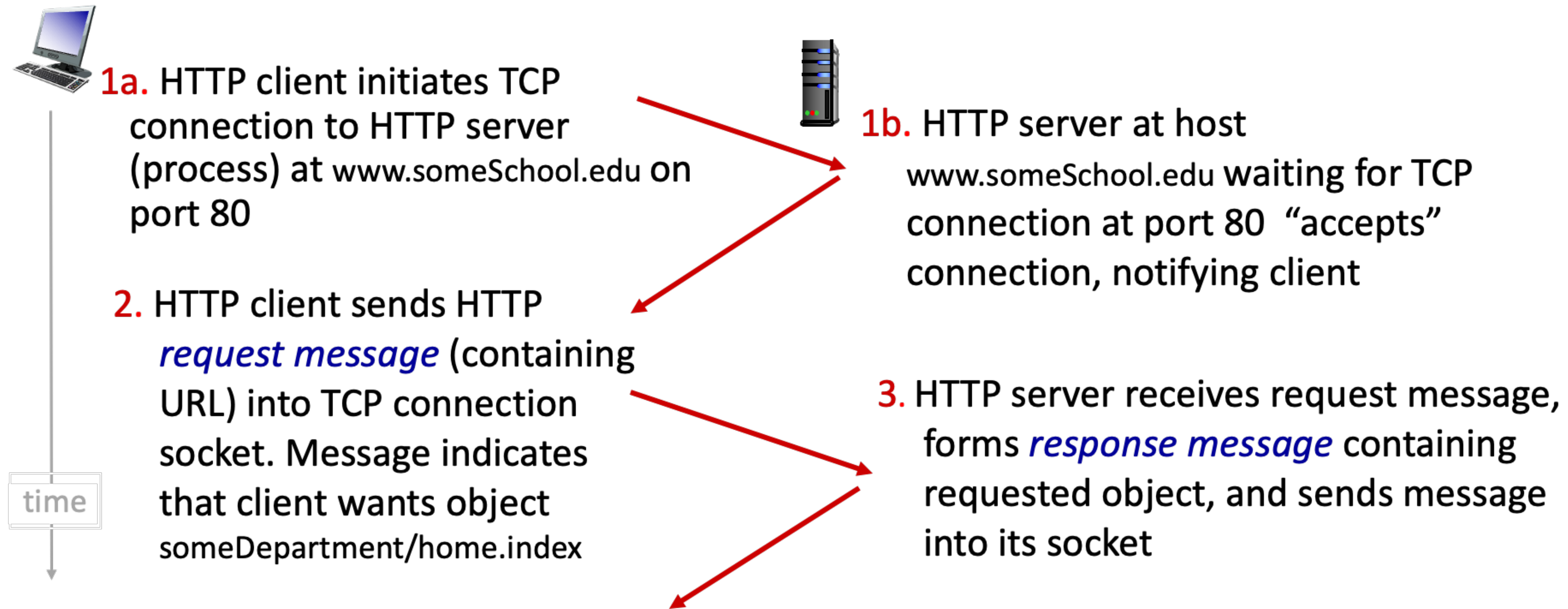
⇒ Downloading multiple objects requires multiple connections

- **Persistent HTTP**

- TCP connection opened
- **Multiple objects** can be **sent over a single TCP connection** between client, and that server
- TCP connection is closed

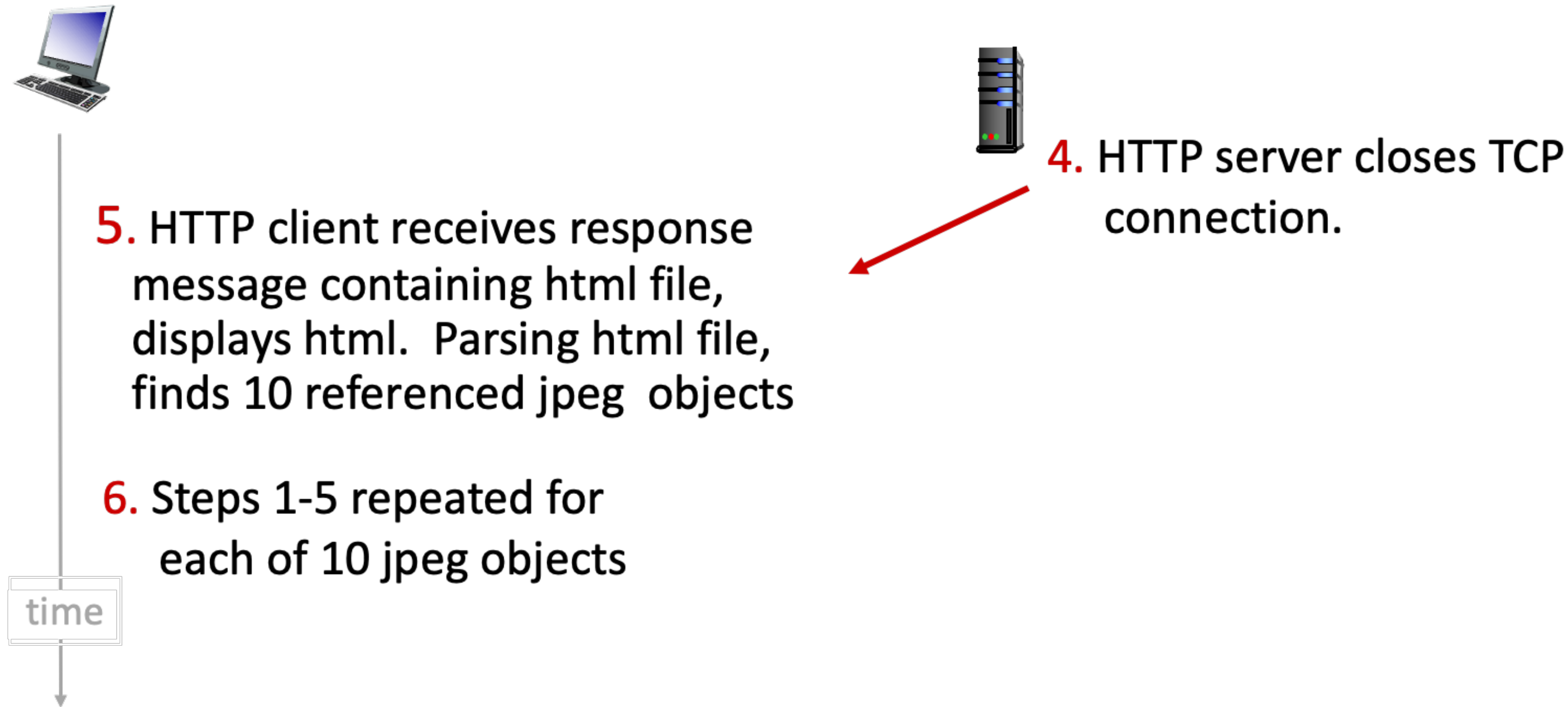
e.g., Non-persistent HTTP

- Suppose user enters URL : `www.someSchool.edu/someDepartment/home.index`
(containing text, references to 10 jpeg images)



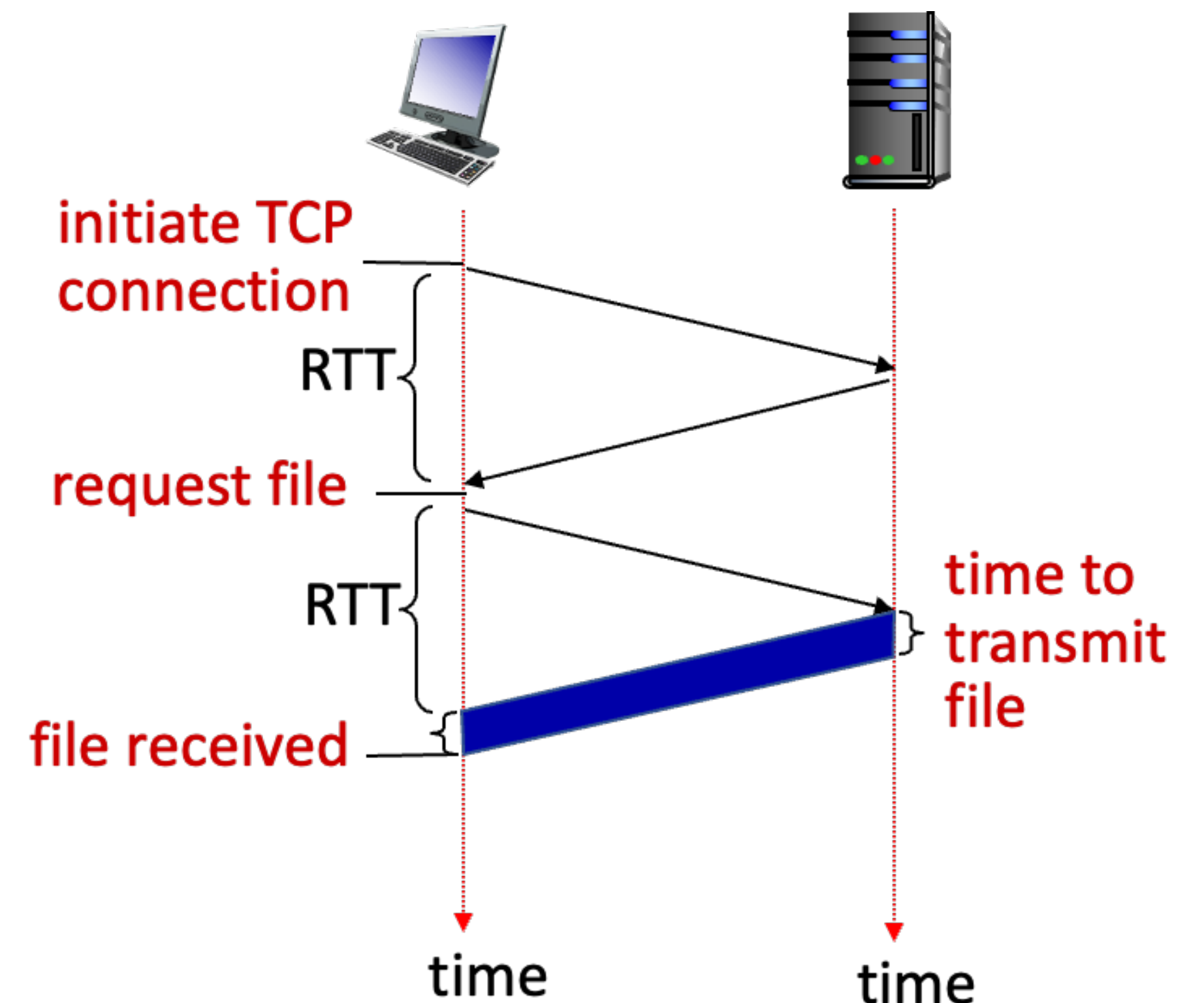
e.g., Non-persistent HTTP

- Suppose user enters URL : `www.someSchool.edu/someDepartment/home.index`
(containing text, references to 10 jpeg images)



Non-persistent HTTP : response time

- RTT (definition) : time for a small packet to travel from client to server and back
- HTTP response time (per object):
 - One RTT to initiate TCP connection
 - One RTT for HTTP request and first few bytes of HTTP response to return
 - Object/file transmission time



Non-persistent HTTP response time = 2RTT + file transmission time

Persistent HTTP (HTTP 1.1)

- Non-persistent issues:
 - Requires 2RTTs per object
 - OS overhead for each TCP connection
 - Browsers often open multiple parallel TCP connections to fetch referenced objects in parallel
- Persistent HTTP (HTTP 1.1):
 - Server **leaves connection open** after sending response
 - Subsequent HTTP messages between the same client/server are sent over the open connection
 - Client sends requests as soon as it encounters a referenced object
 - As little as one RTT for all the referenced objects (cutting response time in half)

HTTP request message

- Two types of HTTP messages: request, response
- HTTP request message

- ASCII (human readable format)

request line (GET, POST,
HEAD commands)

requesting object

carriage return character
line-feed character

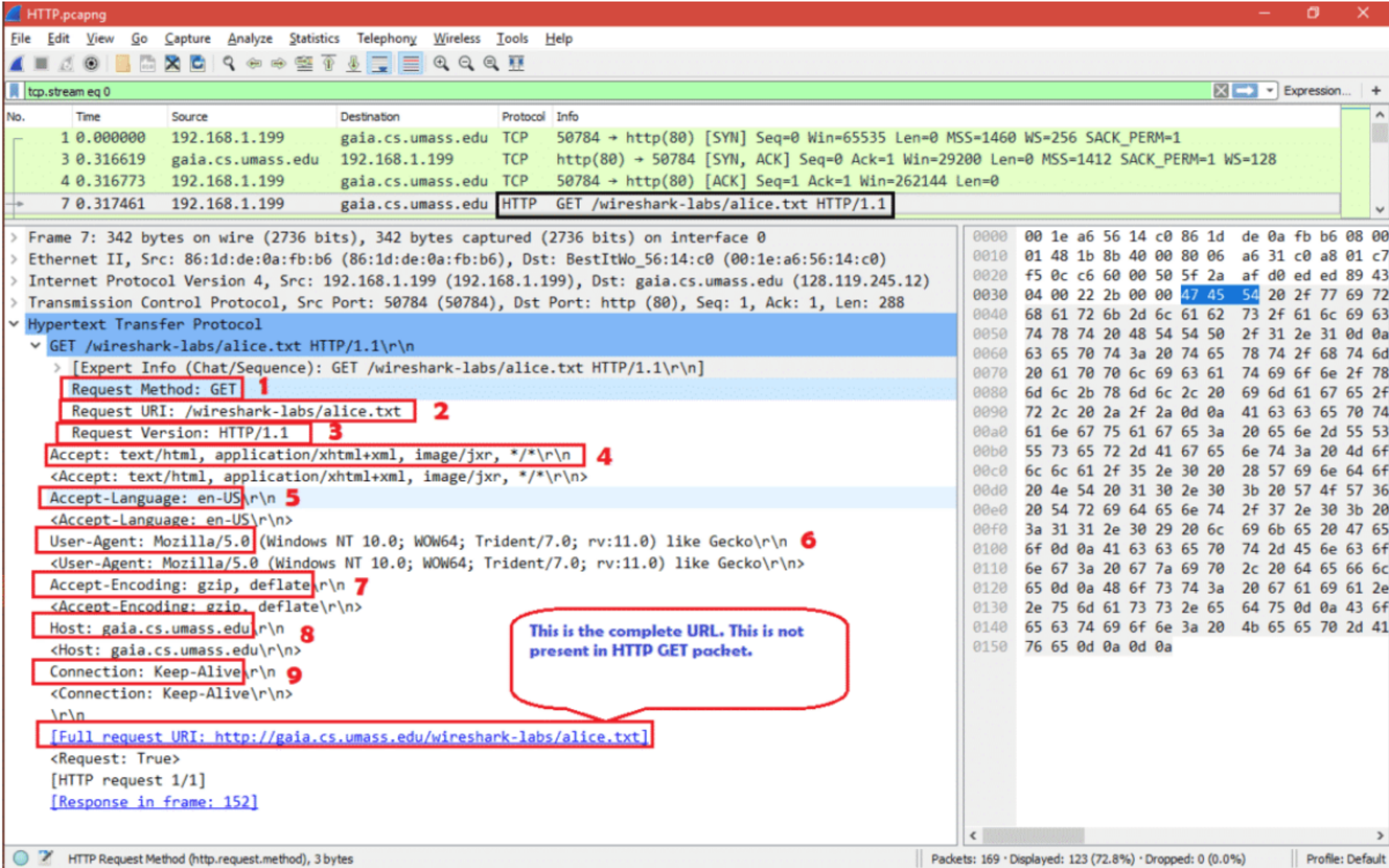
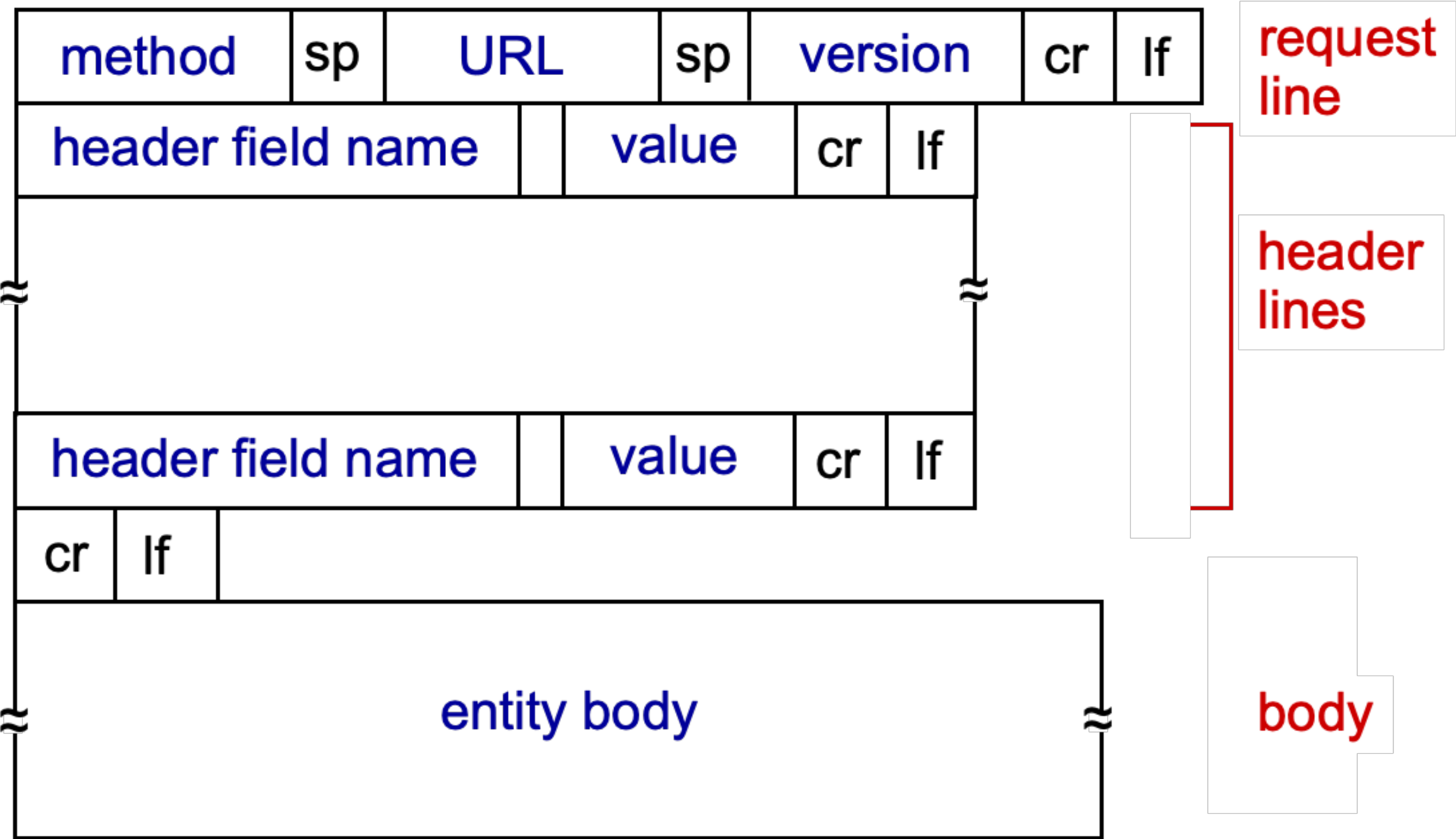
header
lines

```
GET /index.html HTTP/1.1\r\n
Host: www-net.cs.umass.edu\r\n
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X
10.15; rv:80.0) Gecko/20100101 Firefox/80.0 \r\n
Accept: text/html,application/xhtml+xml\r\n
Accept-Language: en-us,en;q=0.5\r\n
Accept-Encoding: gzip,deflate\r\n
Connection: keep-alive\r\n
\r\n
```

carriage return, line feed
at start of line indicates
end of header lines

HTTP request message

- General format



Other HTTP request messages

- **Post method**

- Web page often includes form input
- User input is sent from client to server in entity body of HTTP POST request message

```
POST /inventory-check.cgi HTTP/1.1
Host: www.joes-hardware.com
Content-Type: text/plain
Content-Length: 18

Item=bandsaw
```

Joe's Inventory Check

Check item

- **Get method** (for sending data to server)

- Include user data in the URL field of HTTP GET request message (following a '?')

`www.somesite.com/animalsearch?monkeys&banana`

Other HTTP request messages

- **HEAD method**

- Is identical to HTTP GET but only retrieves the HTTP RESPONSE HEADER of the specified URL, omitting the representation of the source.
- Can be used for testing resource availability

- **PUT method**

- Upload new files (objects) to server
- Completely replaces a file that exists at the specified URL with content in the entity body of POST HTTP request message

HTTP response message

status line (protocol
status code status phrase)

header
lines

data, e.g., requested
HTML file

```
HTTP/1.1 200 OK
Date: Tue, 08 Sep 2020 00:53:20 GMT
Server: Apache/2.4.6 (CentOS)
       OpenSSL/1.0.2k-fips PHP/7.4.9
       mod_perl/2.0.11 Perl/v5.16.3
Last-Modified: Tue, 01 Mar 2016 18:57:50 GMT
ETag: "a5b-52d015789ee9e"
Accept-Ranges: bytes
Content-Length: 2651
Content-Type: text/html; charset=UTF-8
\r\n
data data data data data ...
```

HTTP response status codes

- Status code appears in the 1st line in server-to-client response message
- Some sample codes:
 - **200 OK** (request succeeded)
 - Following this code, the requested object is included in this message
 - **301 Moved Permanently** (requested object moved)
 - New location is specified in this message (in Location: field)
 - **400 Bad Request** (request message is not understood by server)
 - **404 Not Found** (requested object is not found on this server)
 - **505 HTTP Version Not Supported**

Trying out HTTP (client side)

- netcat to your favorite Web server

% nc -c -v cse.kangwon.ac.kr 80 (for Mac)

> ncat -C cse.kangwon.ac.kr 80 (for Windows)

- Open TCP connection to port 80 (default HTTP server port)
- Anything typed in will be sent to port 80 at cse.kangwon.ac.kr

- Type in a GET HTTP request

GET /cse/index.do HTTP/1.1

Host: cse.kangwon.ac.kr

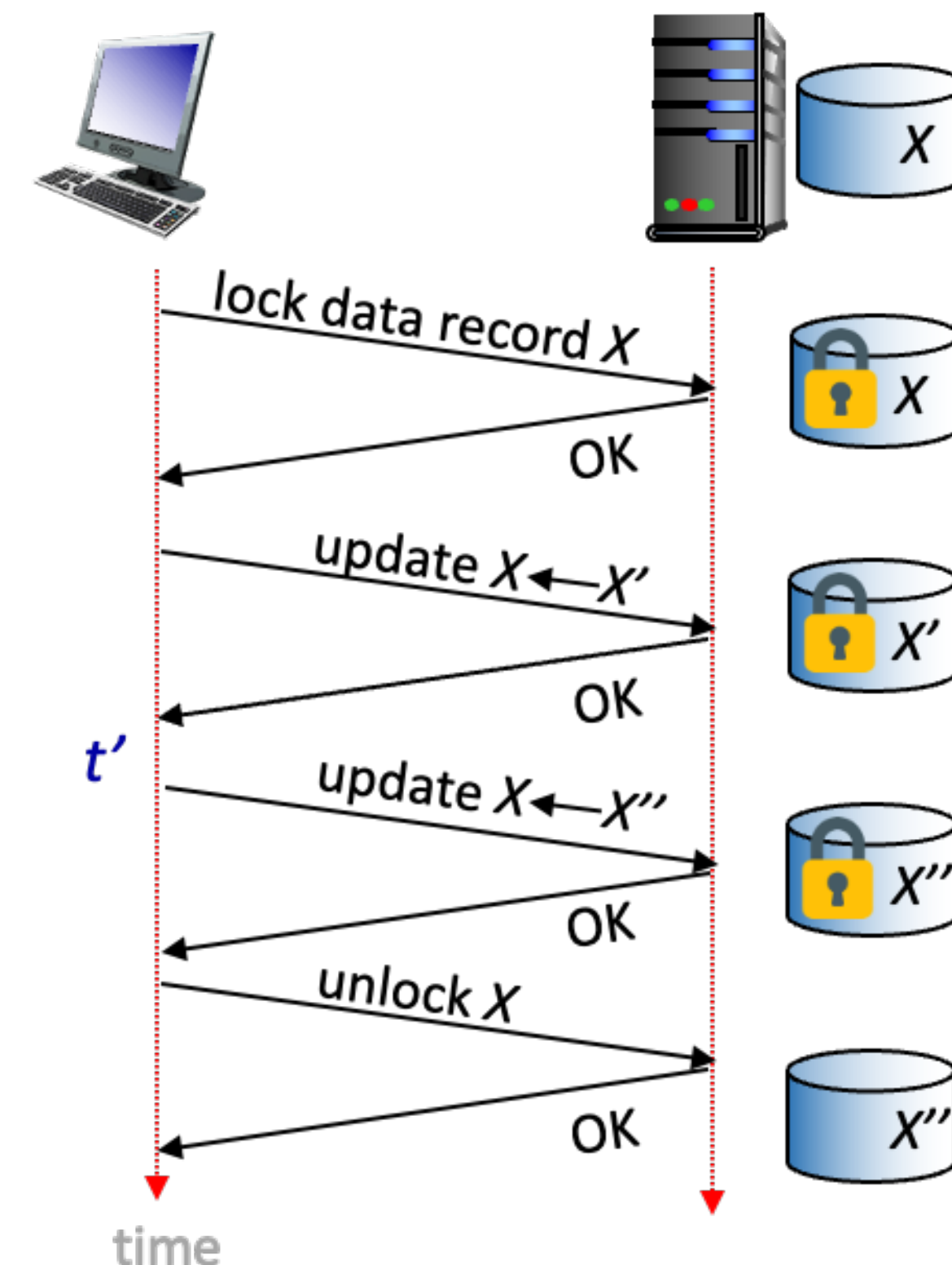
- by typing this in (hit carriage return twice), you send this minimal (but complete) GET request to HTTP server

- Look at response message sent by HTTP server
(or use Wireshark to look at the captured HTTP request / response)

Maintaining user/server state: cookies

- Recall: HTTP GET/response interaction is **stateless**
- No notion of multi-step exchanges of HTTP messages to complete a Web “transaction”
 - No need for client/server to track “state” of multi-step exchange
 - All HTTP requests are independent of each other
 - No need for client/server to “recover” from a partially-completed-but-never-completely-completed transaction

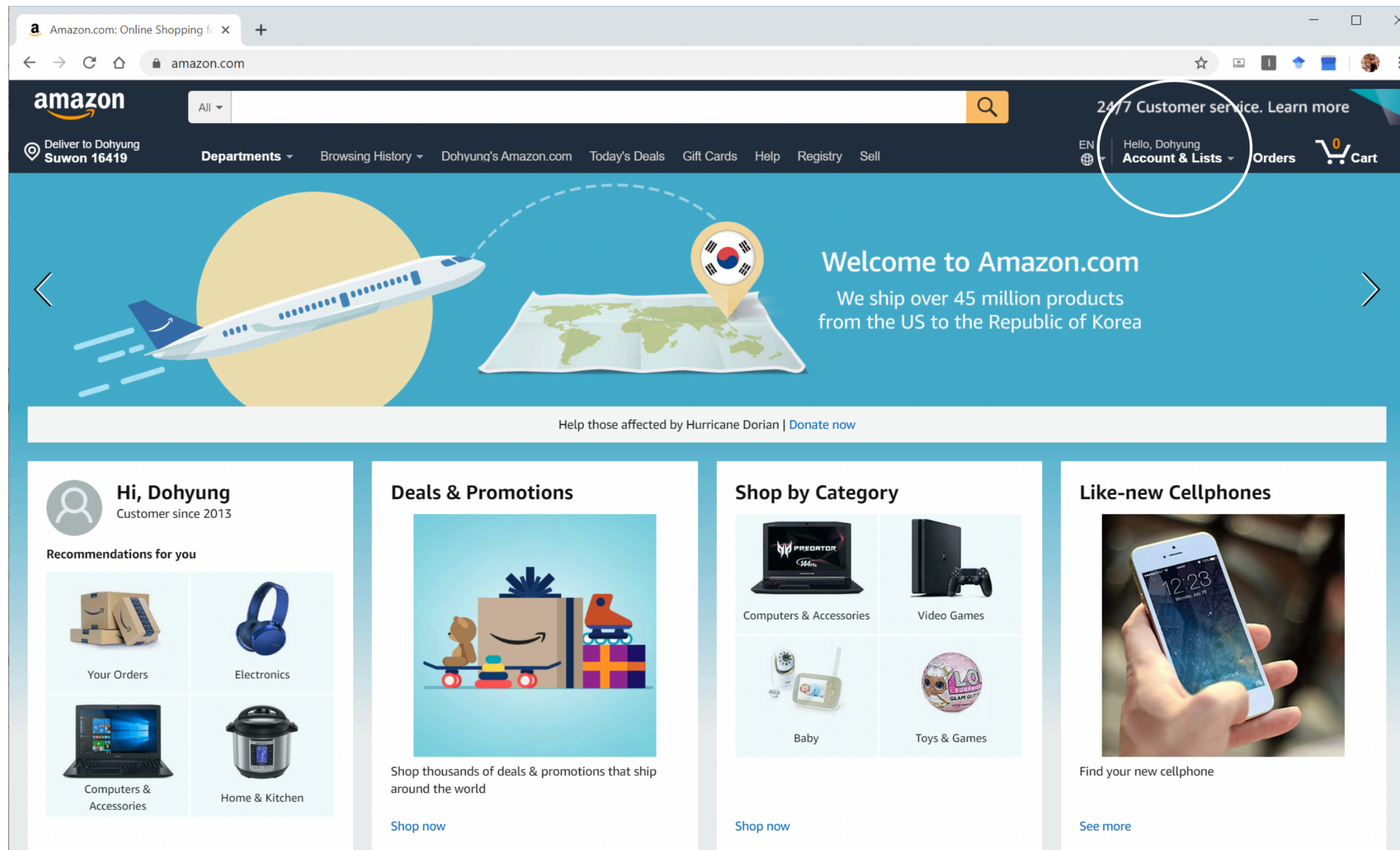
A **stateful protocol** : client makes two changes to X , or none at all



Q) What happens if network connection or client crashes at t' ?

Maintaining user/server state: cookies

- Web sites and client browser use cookies to maintain some state between transaction



Maintaining user/server state: cookies

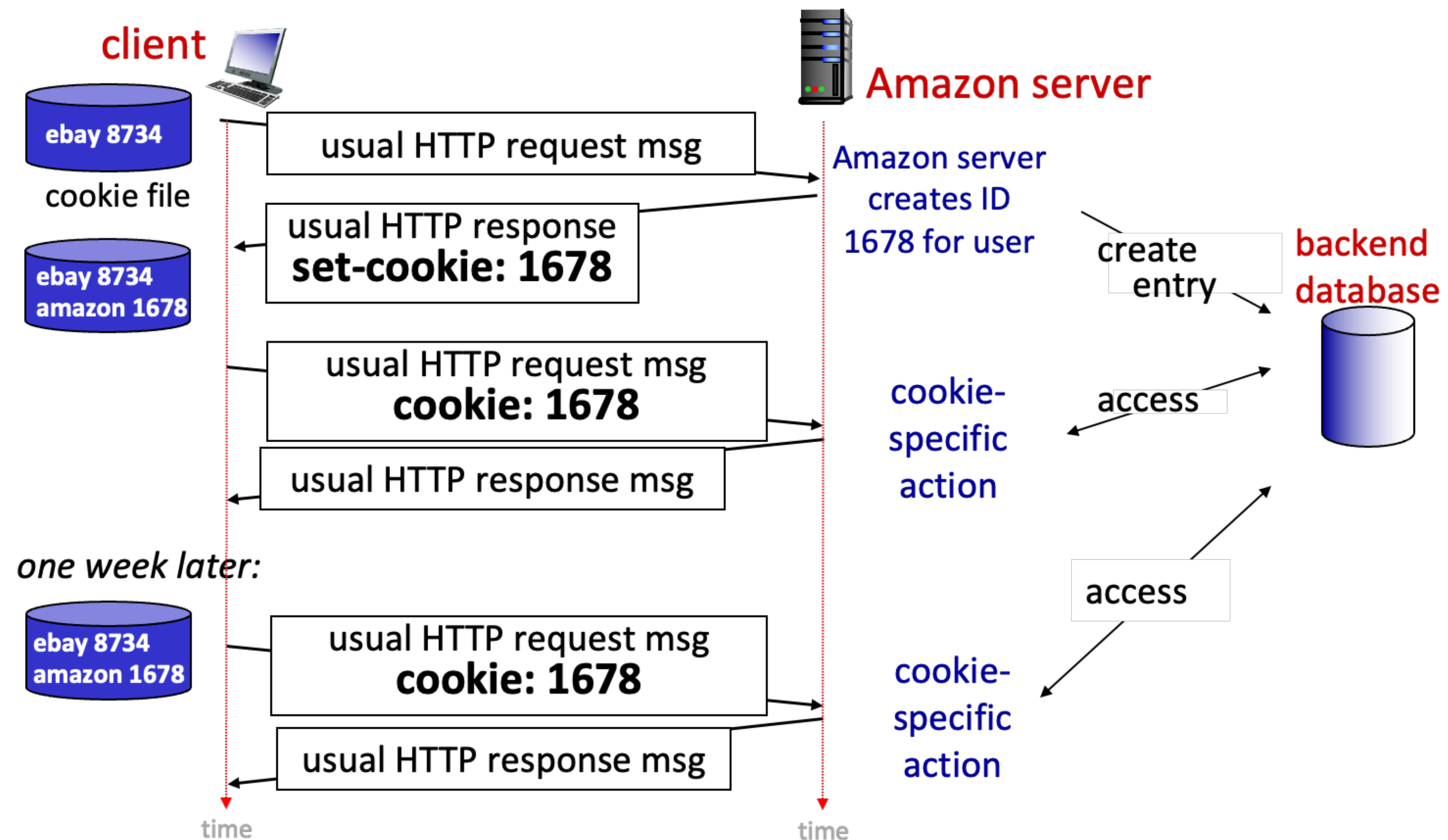
- Web sites and client browser use cookies to maintain some state between transaction
- Four components:
 - Cookie header line of HTTP response message
 - Cookie header line in next HTTP request message
 - Cookie file kept on user's host, managed by user's browser
 - Back-end database at Web site

Maintaining user/server state: cookies

- Web sites and client browser use cookies to maintain some state between transaction
- e.g., cookie example
 - Susan uses a browser on laptop, visits specific e-commerce site for the first time
 - When the initial HTTP request arrives at site, site creates:
 - Unique ID (aka “cookie”)
 - Entry in backend database for the ID
 - Subsequent HTTP requests from Susan to this site will contain cookie ID value, allowing site to “identify” Susan

Maintaining user/server state: cookies

- Web sites and client browser use cookies to maintain some state between transaction



HTTP cookies: comments

- What cookies can be used for

- Authorization
- Shopping carts
- Recommendations
- User session state (Web e-mail)

Cookies and privacy

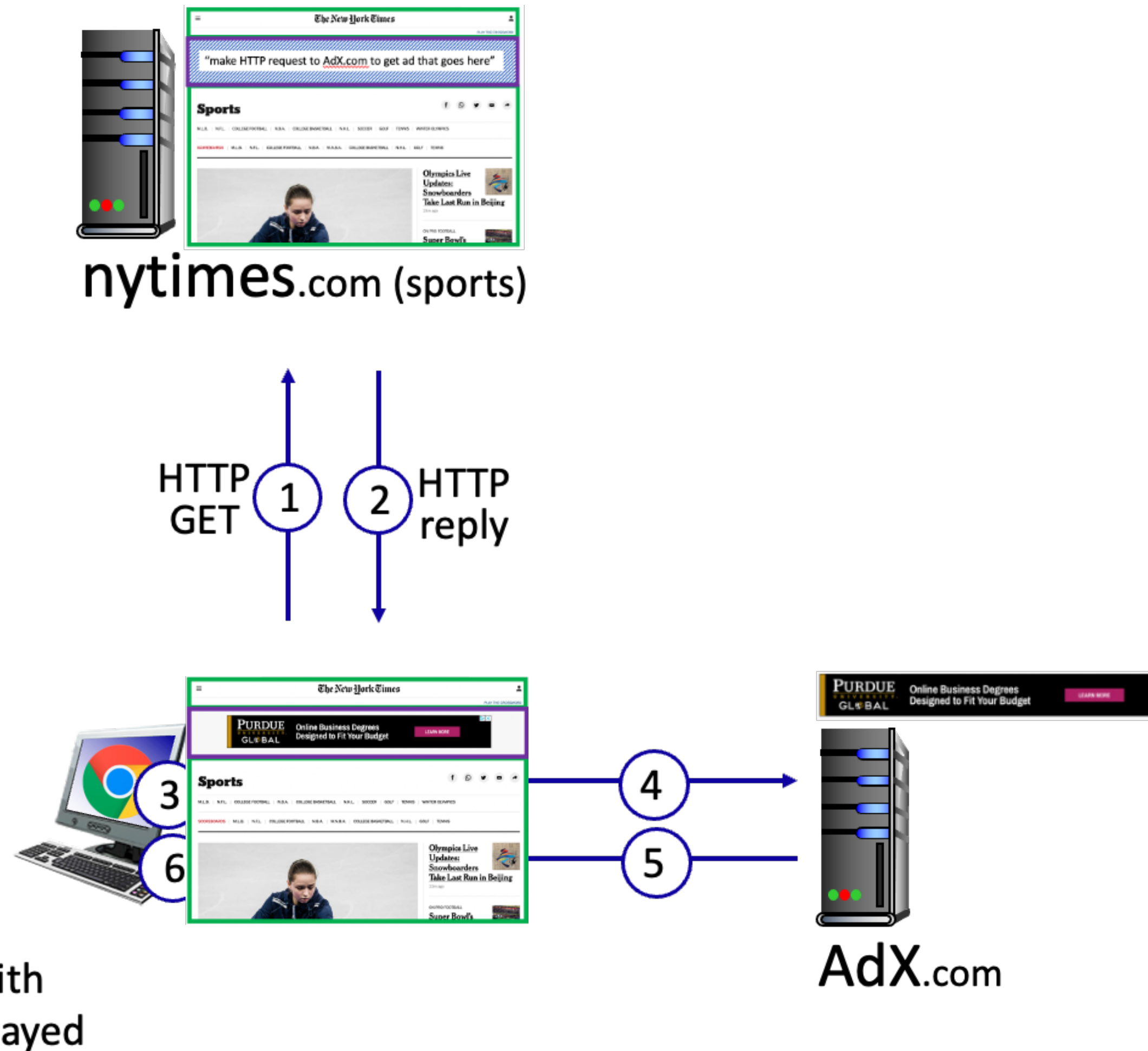
- cookies permit sites to learn a lot about you on their site
- Third party persistent cookies (tracking cookies) allow common identity (cookie value) to be tracked across multiple websites

- Challenge: How to keep state?

- At protocol endpoints: maintain state at sender/receiver over multiple transactions
- In messages: cookie in HTTP messages carry state

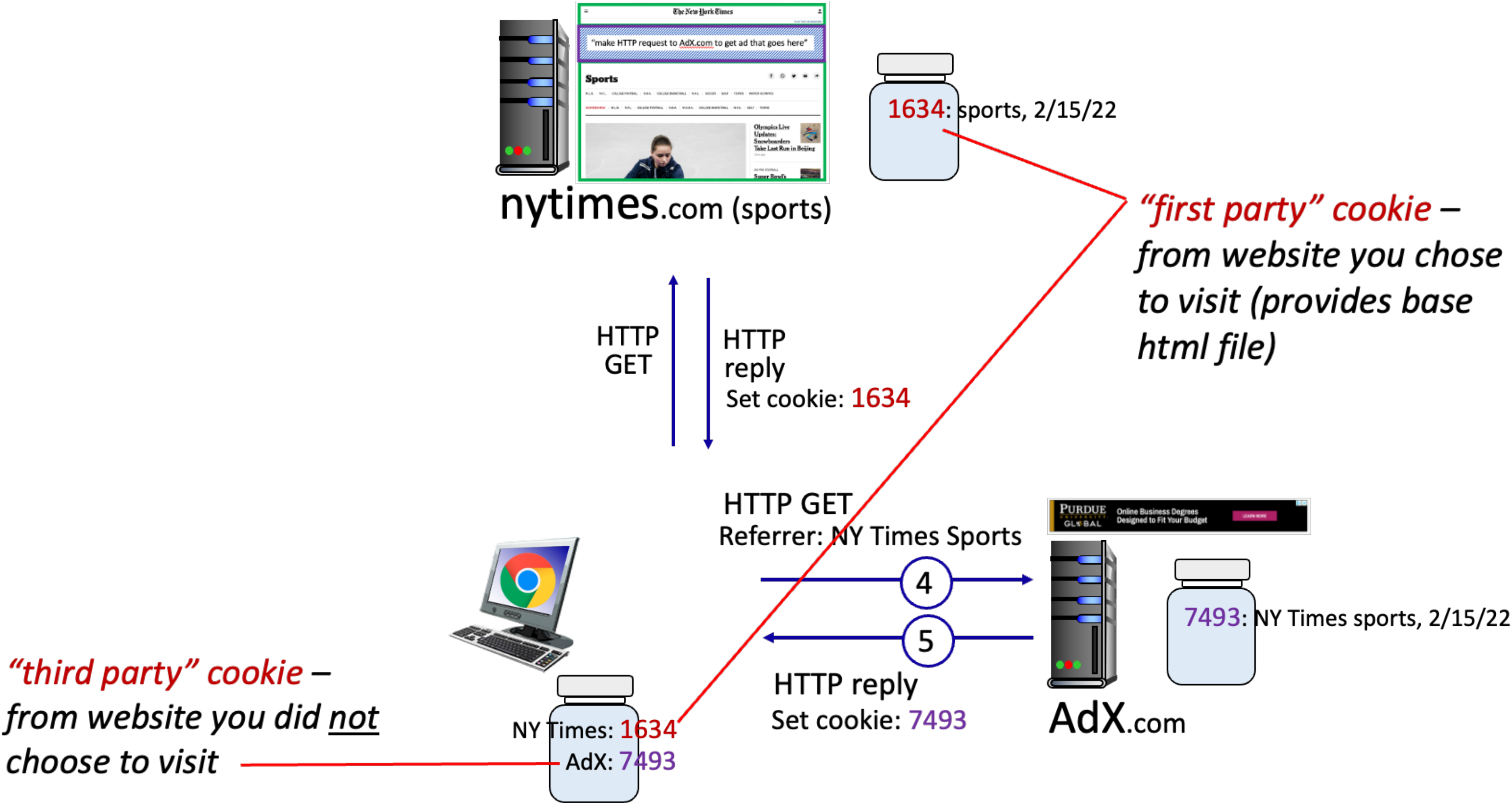
e.g., displaying a NY Times web page

- 1 GET base html file
- 2 from nytimes.com
- 4 fetch ad from
- 5 AdX.com
- 7 display composed page

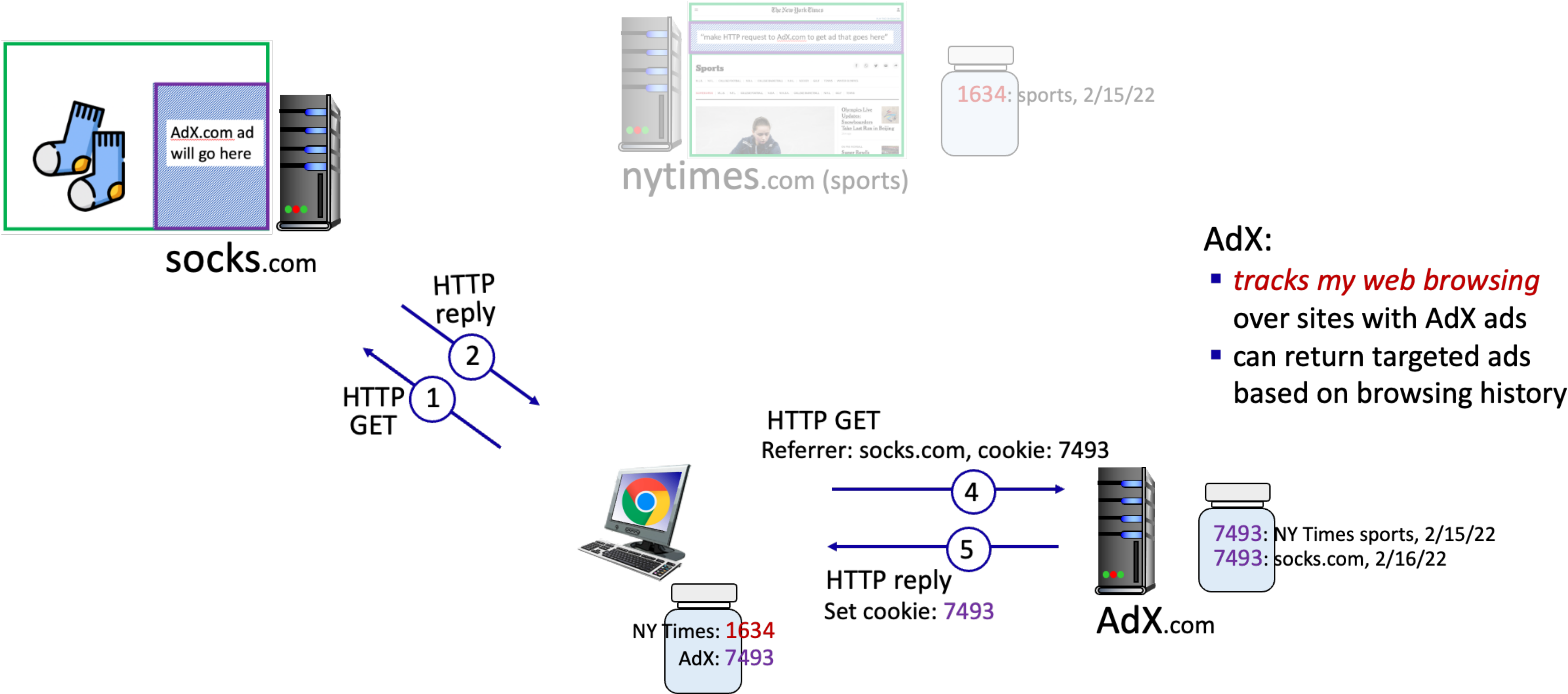


NY times page with
embedded ad displayed

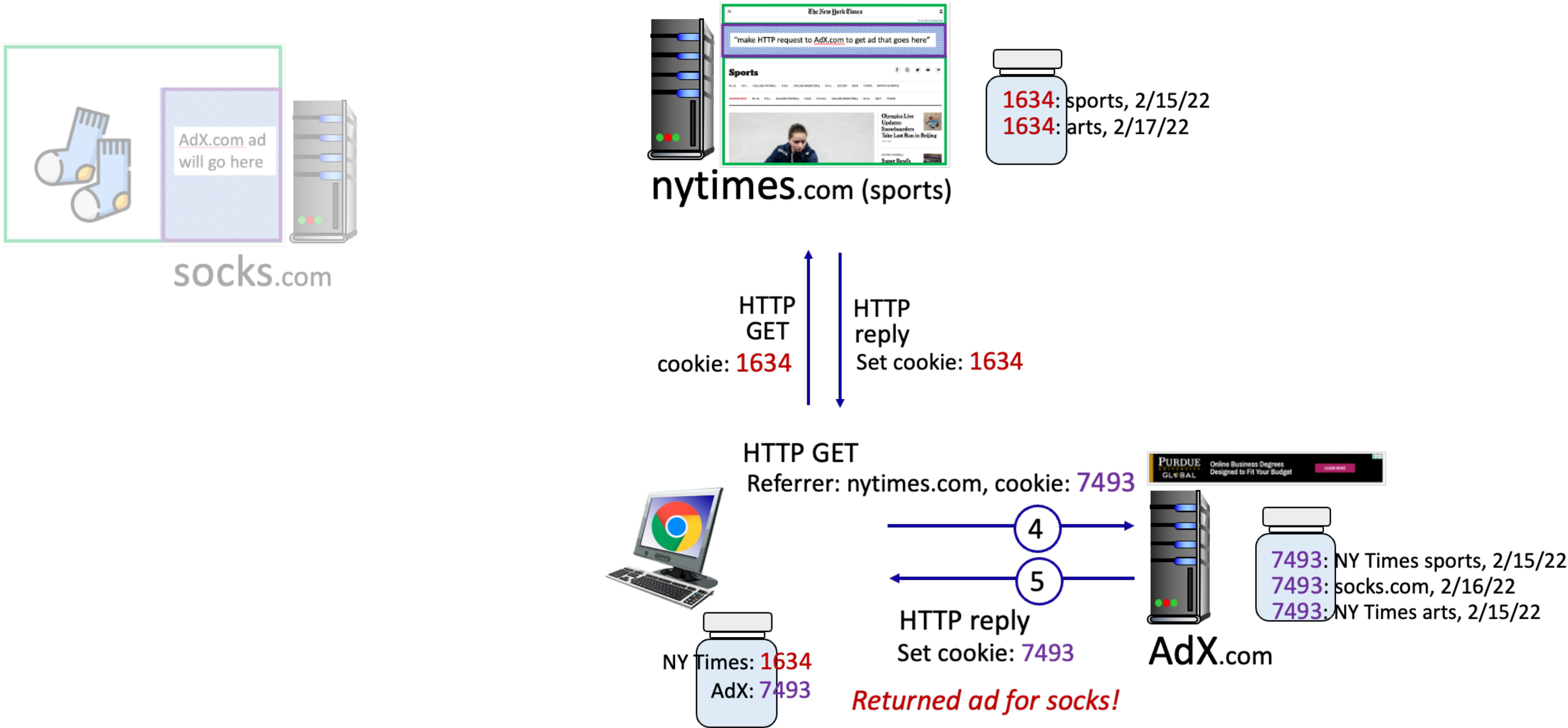
e.g., displaying a NY Times web page



e.g., displaying a NY Times web page



e.g., displaying a NY Times web page (one day layer)



Cookies: tracking a user's browsing behavior

- Cookies can be used to
 - Track user behavior on a given website (first party cookie)
 - Track user behavior across multiple websites (third party cookies) without user ever choosing to visit tracker site (!)
 - Tracking may be invisible to user
 - Rather than displayed ad triggering HTTP GET to tracker, could be an invisible link

Third party tracking via cookies

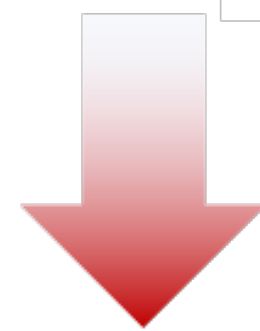
- ✓ Disabled by default in Firefox, Safari browsers
- ✓ To be disabled in Chrome browser in 2023

GDPR (EU General Data Protection Regulation)

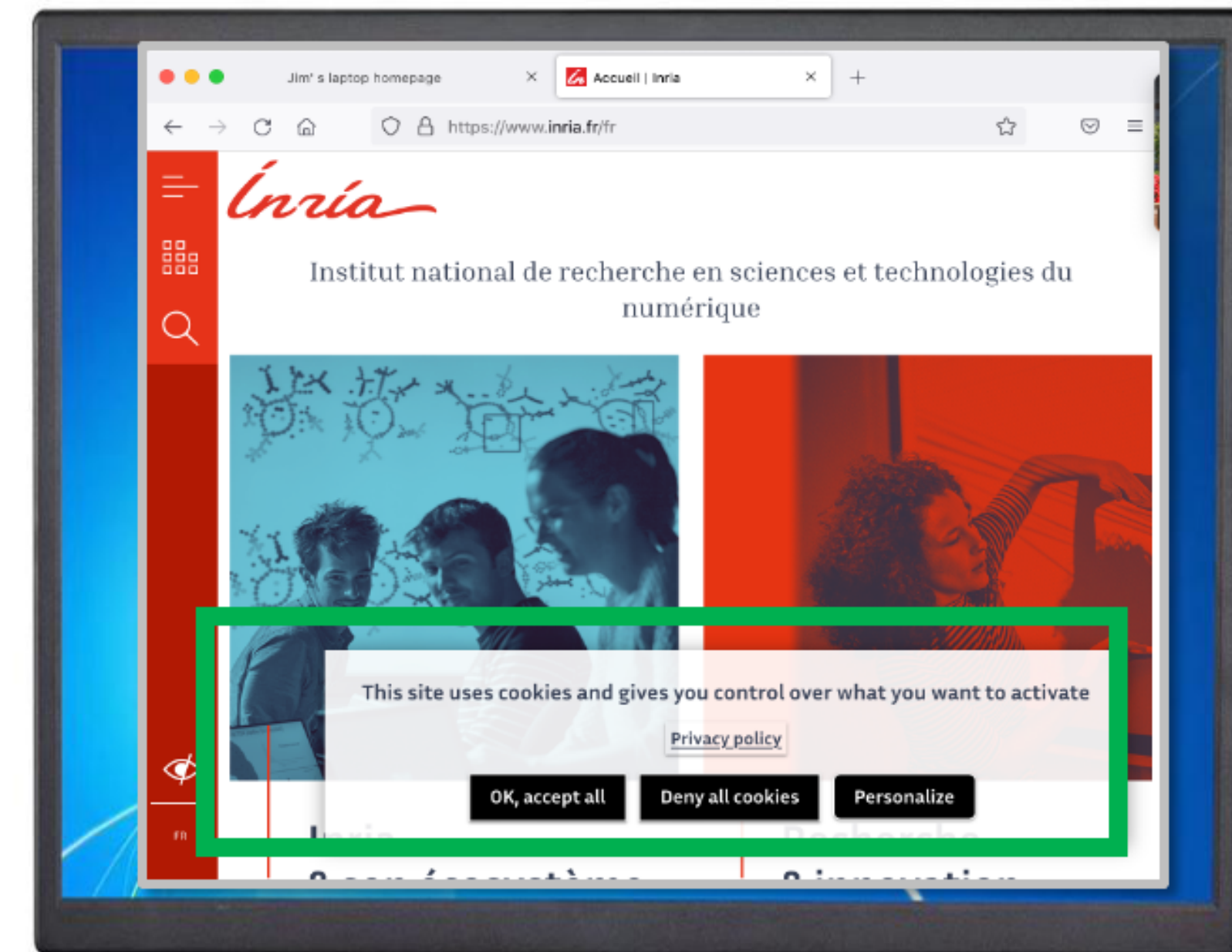
“Natural persons may be associated with online identifiers [...] such as internet protocol addresses, cookie identifiers or other identifiers [...].

This may leave traces which, in particular when combined with unique identifiers and other information received by the servers, may be used to create profiles of the natural persons and identify them.”

GDPR, recital 30 (May 2018)



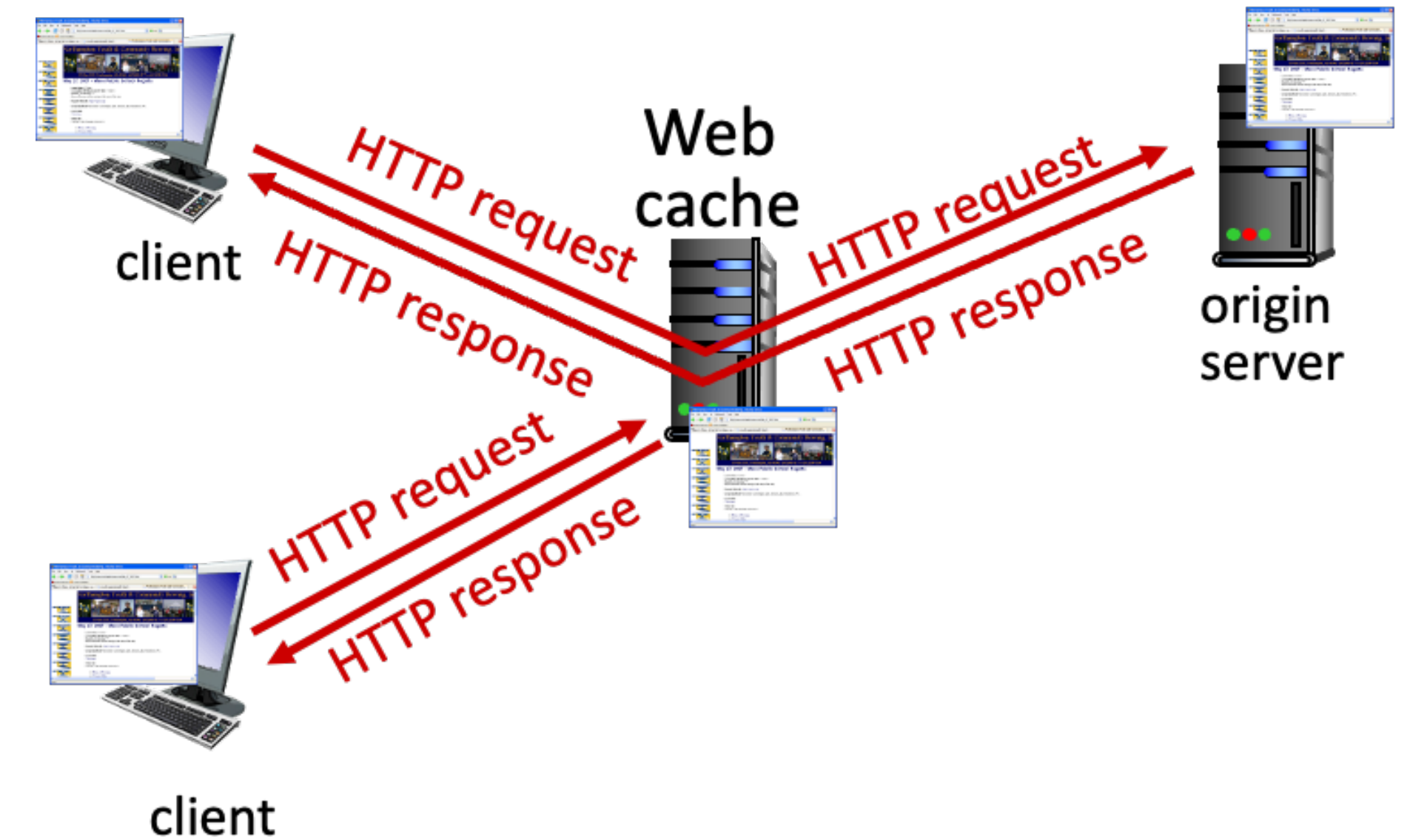
when cookies can identify an individual, cookies are considered personal data, subject to GDPR personal data regulations



User has explicit control over whether or not cookies are allowed

Web caches

- Objective
 - Satisfy client requests without involving the origin server
- Procedures
 - User configures browser to point to a (local) Web cache
 - Browser sends all HTTP requests to cache
 - if object in cache: cache returns object to client
 - else: cache requests object from origin server, caches received objects then returns object to client



Web caches (aka proxy servers)

- Web cache acts as both client and server
 - Server for original requesting client
 - Client to origin server
- Server tells cache about object's allowable caching in response header

```
Cache-Control: max-age=<seconds>
```

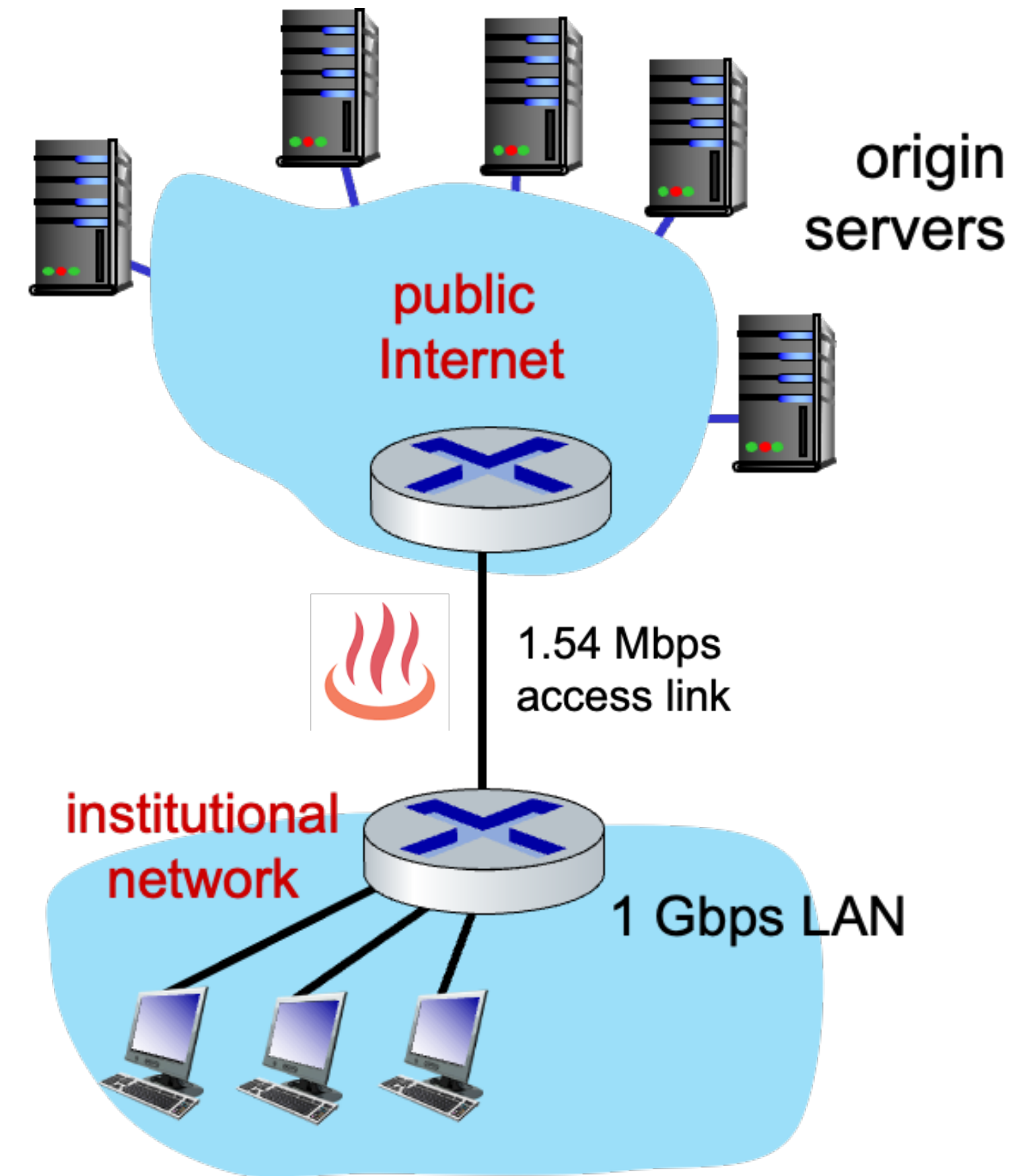
```
Cache-Control: no-cache
```

Why Web caching?

- ✓ Reduce response time for client request : cache is closer to client
- ✓ Reduce traffic on an institution's access link
- ✓ Internet is dense with caches: enables “poor” content providers to more effectively deliver content

Caching example

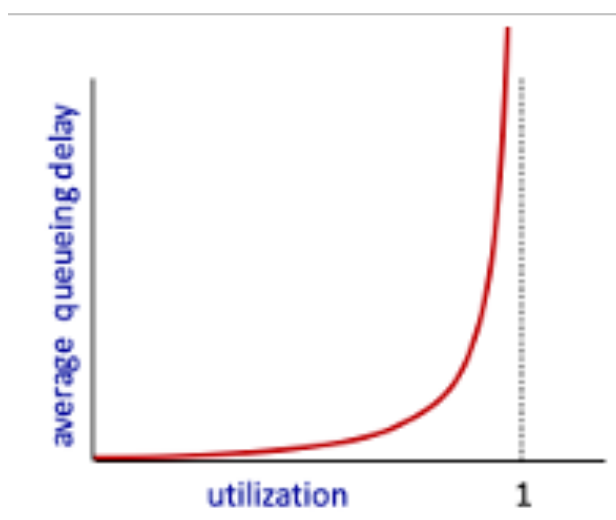
- Scenario
 - Access link rate: 1.54Mbps
 - RTT from institutional router to server: 2 secs
 - Web object size: 100K bits
 - Average request rate from browsers to origin server: 15/sec
 - Average rate to browsers: 1.5Mbps



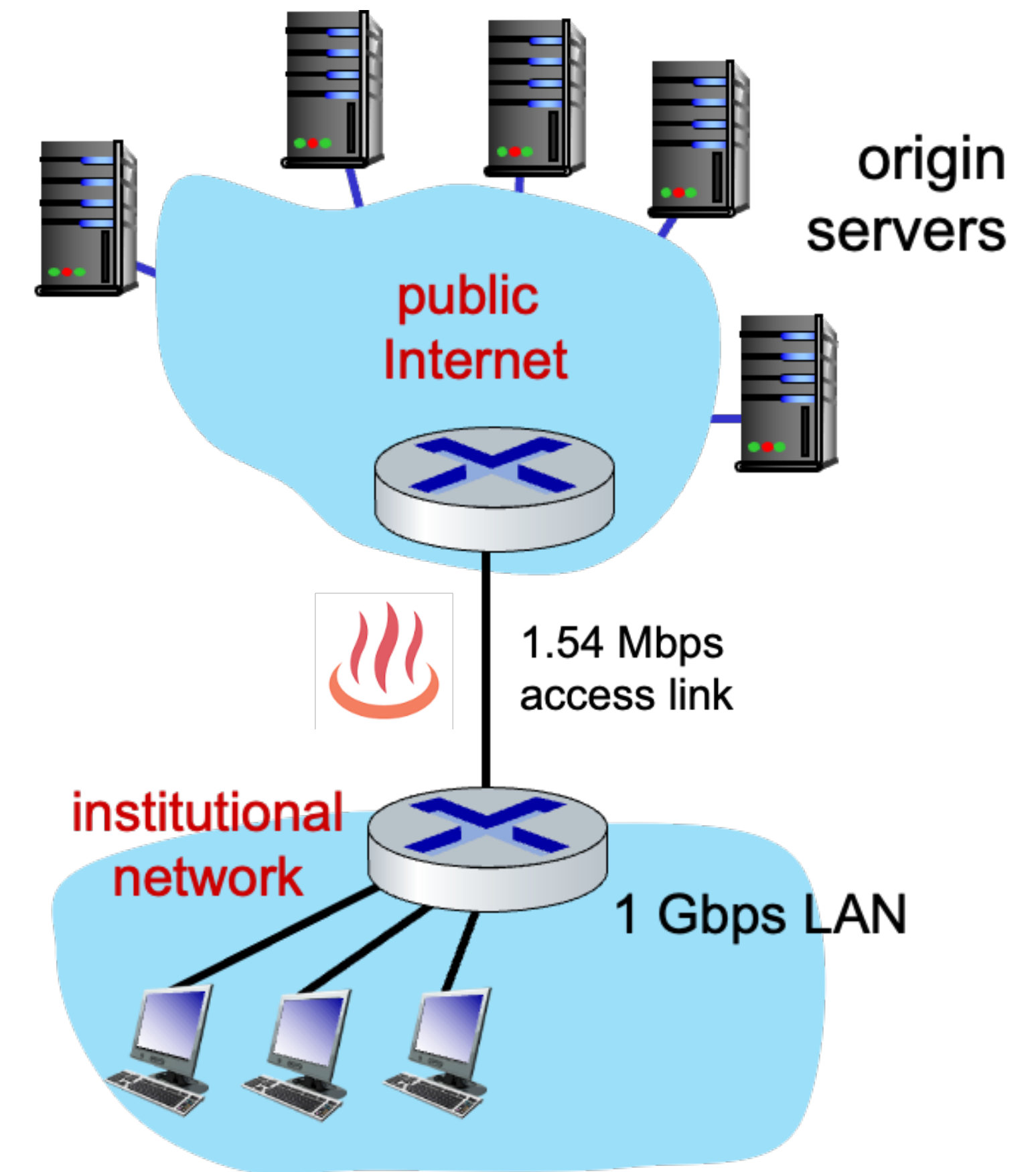
Caching example

- Performance

- Access link utilization = $1.50 / 1.54 = 0.97$
- LAN utilization: $1.50\text{Mbps} / 1\text{Gbps} = 0.0015$
- End-to-end delay
= Internet delay + access link delay + LAN delay
= 2 secs + minute + usecs



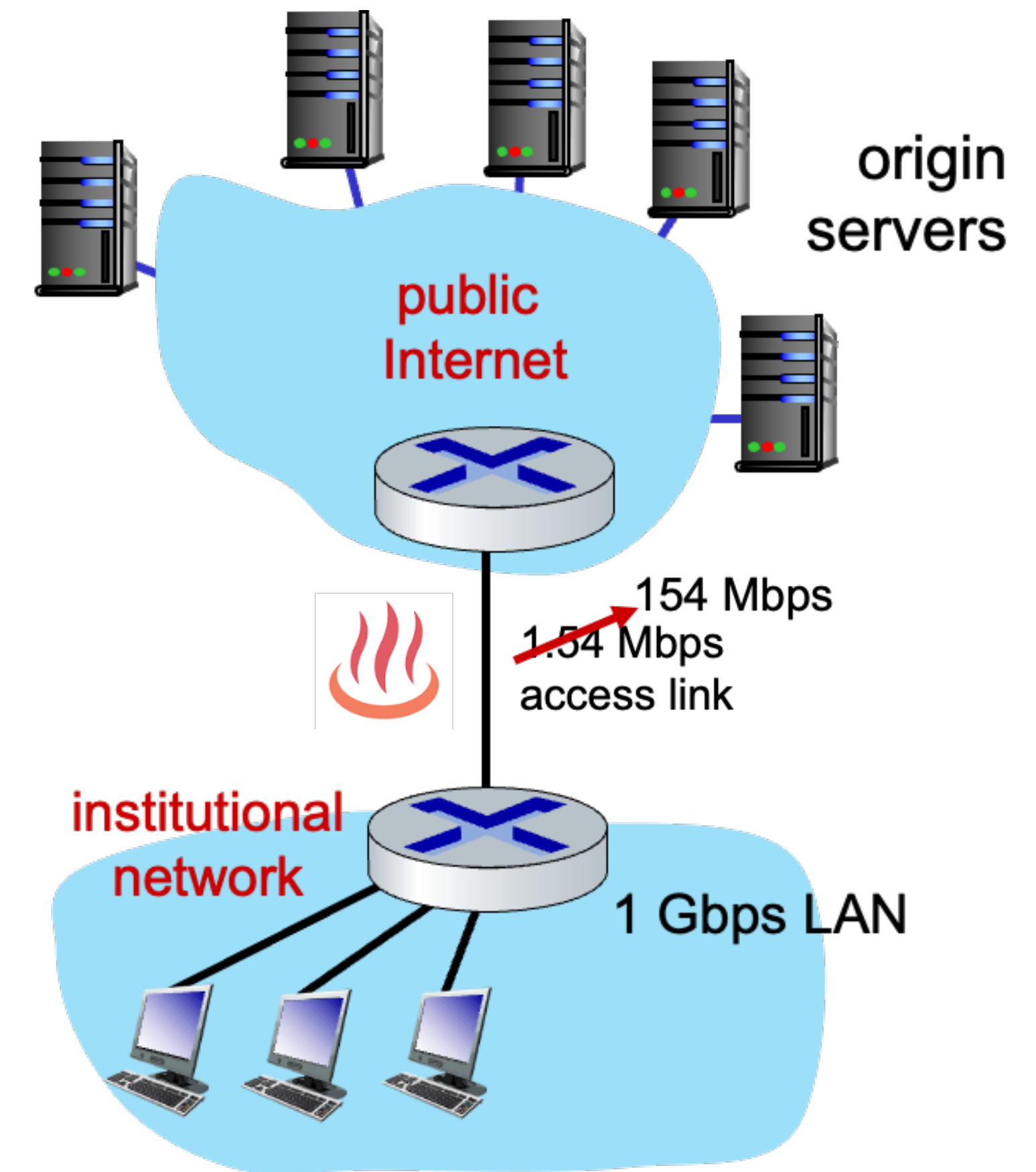
✓ Problem: large queueing delays at high utilization



Solution 1: buy a faster access link

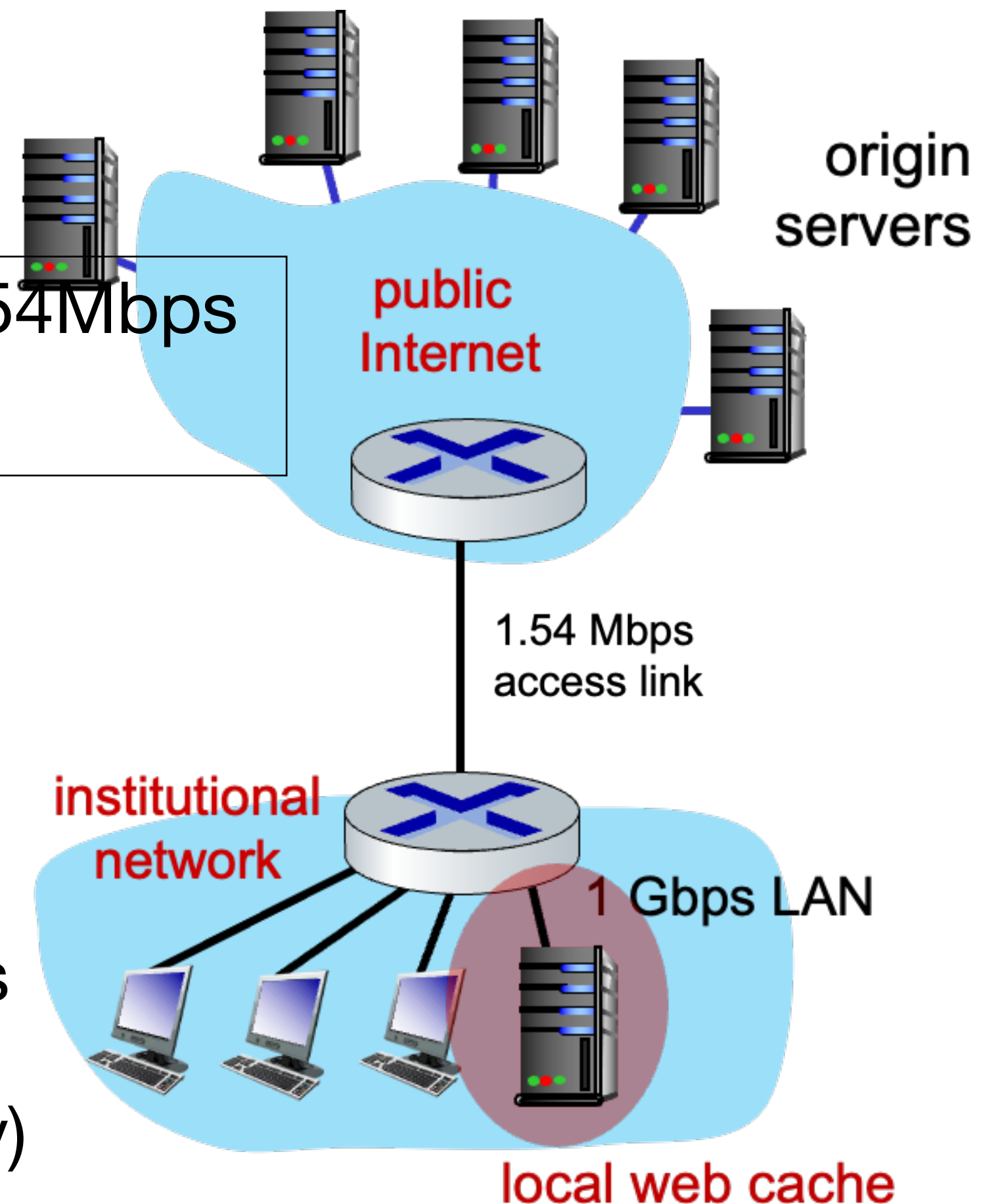
- Access link rate : 1.54Mbps → 154Mbps
- Performance
 - Access link utilization = $1.50 / 154 = 0.0097$
 - LAN utilization: $1.50\text{Mbps} / 1\text{Gbps} = 0.0015$
 - End-to-end delay
 - = Internet delay + access link delay + LAN delay
 - = 2 secs + ~~minute~~ + usecs
 - msecs**

✓ Cost: faster access link is expensive



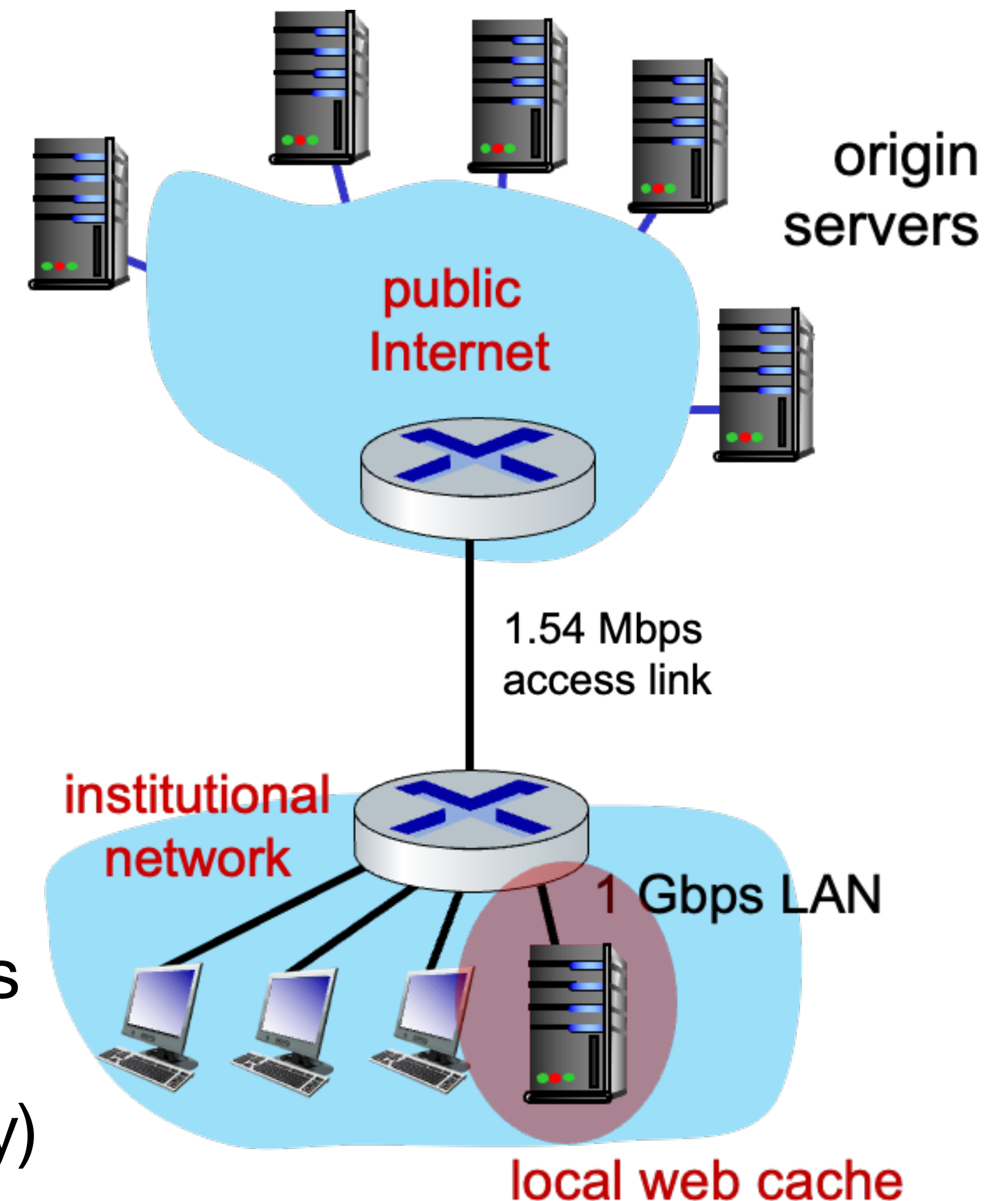
Solution 2: install a web cache

- Network configuration is not changed
- Only a web cache is newly installed
 - Lower average end-to-end delay than with 154Mbps link (and cheaper too!)
- Performance (assume cache hit rate is 0.4)
 - 40% requests served by cache, with low (msecs) delay
 - 60% requests satisfied at origin server
 - Rate to browser over access link: $0.6 \times 1.50\text{Mbps} = 9\text{Mbps}$
 - Access link utilization: $0.9 / 1.54 = 0.58$ (low queueing delay)
 - Average end-to-end delay: $0.6 \times (\text{delay from original server}) + 0.4 \times (\text{delay from cache})$
 $= 0.6 \times (2 + \text{msecs}) + 0.4 \times (\sim\text{msecs}) = \sim 1.2 \text{ secs}$



Solution 2: install a web cache

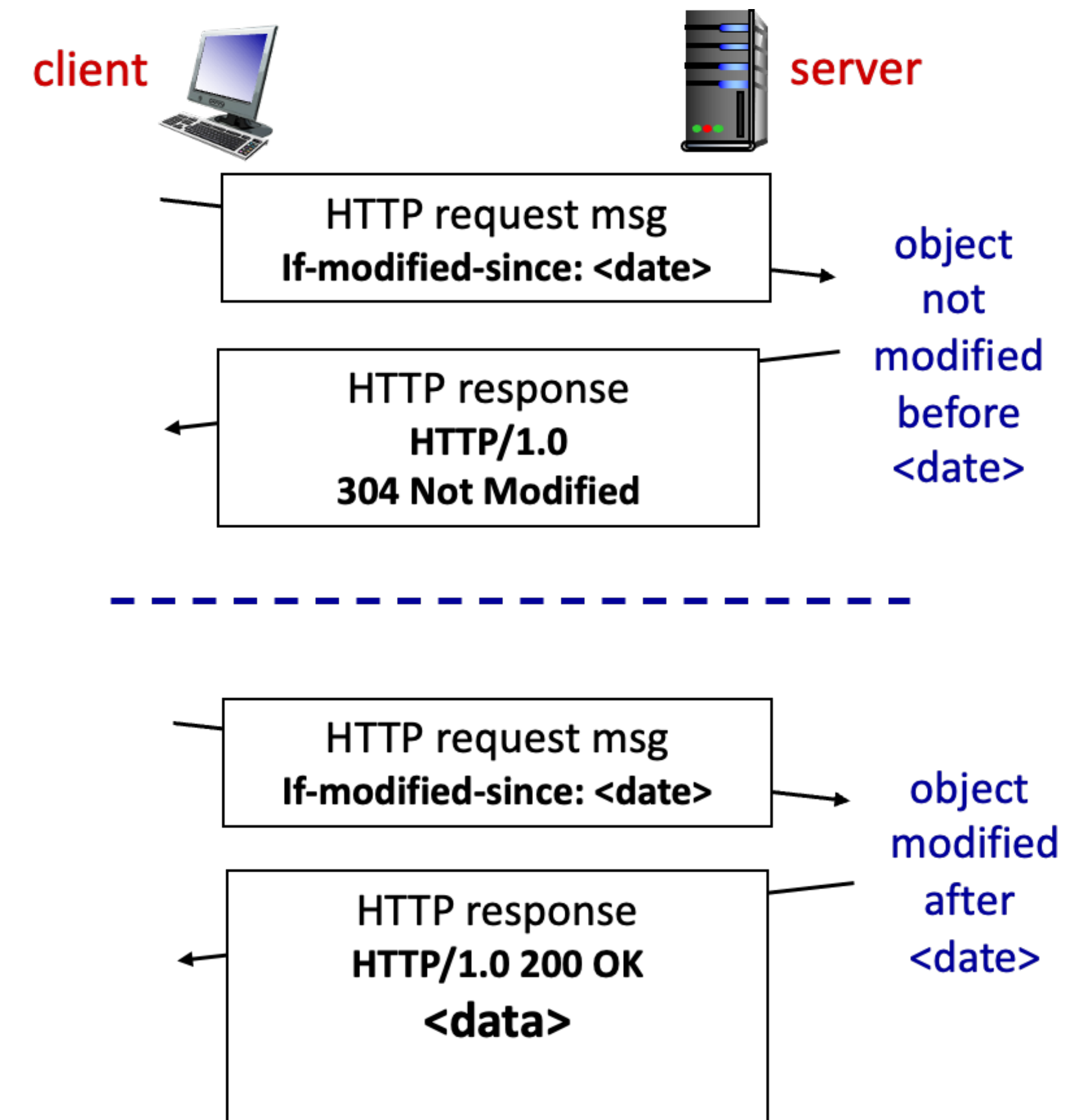
- Network configuration is not changed
- Only a web cache is newly installed
- Performance (assume cache hit rate is 0.4)
 - 40% requests served by cache, with low (msecs) delay
 - 60% requests satisfied at origin server
 - Rate to browser over access link: $0.6 \times 1.50\text{Mbps} = 9\text{Mbps}$
 - Access link utilization: $0.9 / 1.54 = 0.58$ (low queueing delay)
 - Average end-to-end delay: $0.6 \times (2 + \text{msecs}) + 0.4 \times (\sim\text{msecs}) = \sim 1.2 \text{ secs}$



Lower average end-to-end delay than with 154Mbps link (and cheaper too!)

Browser caching: Conditional GET

- Objective
 - Don't send object if browser has up-to-date cached version
 - No object transmission delay (or use of network resources)
- Client: specify date of browser-cached copy in HTTP request
 - **If-modified-since: <date>**
- Server: response contains no object if browser-cached copy is up-to-date
 - **HTTP/1.0 304 Not Modified**



HTTP evolution

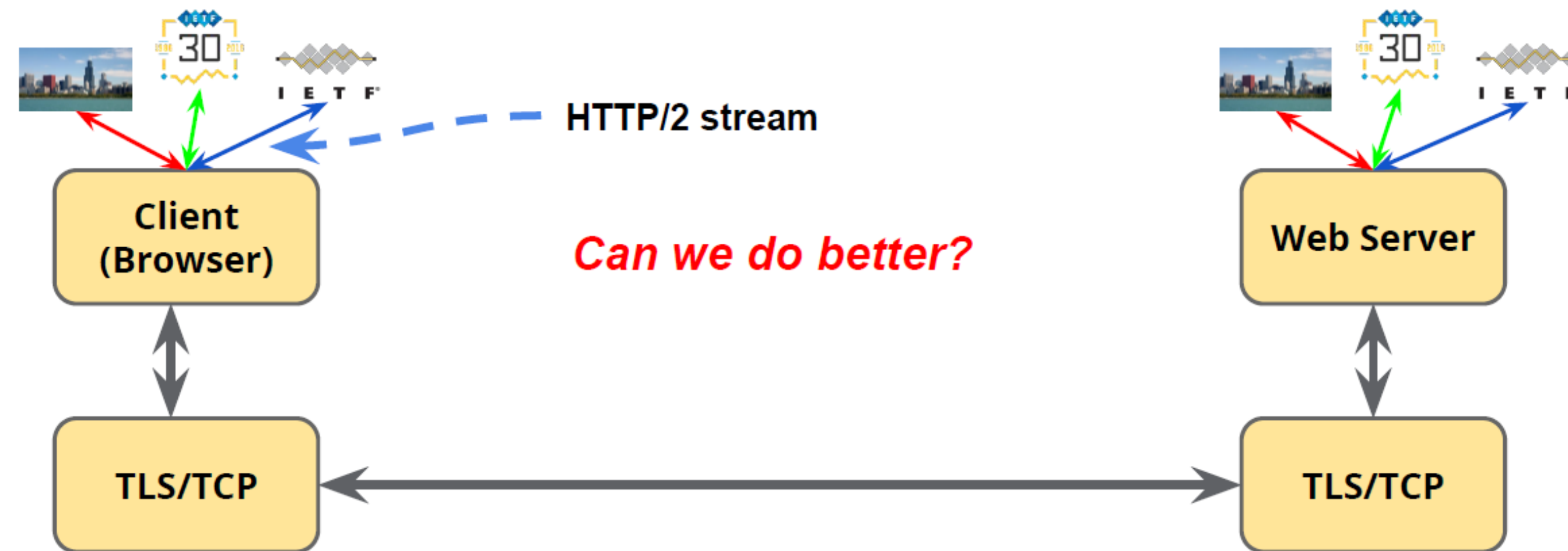
- 1989: WWW
- 1991: HTTP 0.9
- 1996: HTTP 1.0
- 1997: HTTP 1.1
- 2012: HTTP 1.1 bis?
- 2012: SPDY v1
- 2012 ~ 2015: SPDY v2, v3, v3.1, v4 alpha3
- 2015: HTTP/2
- 2023: HTTP/3

HTTP/1



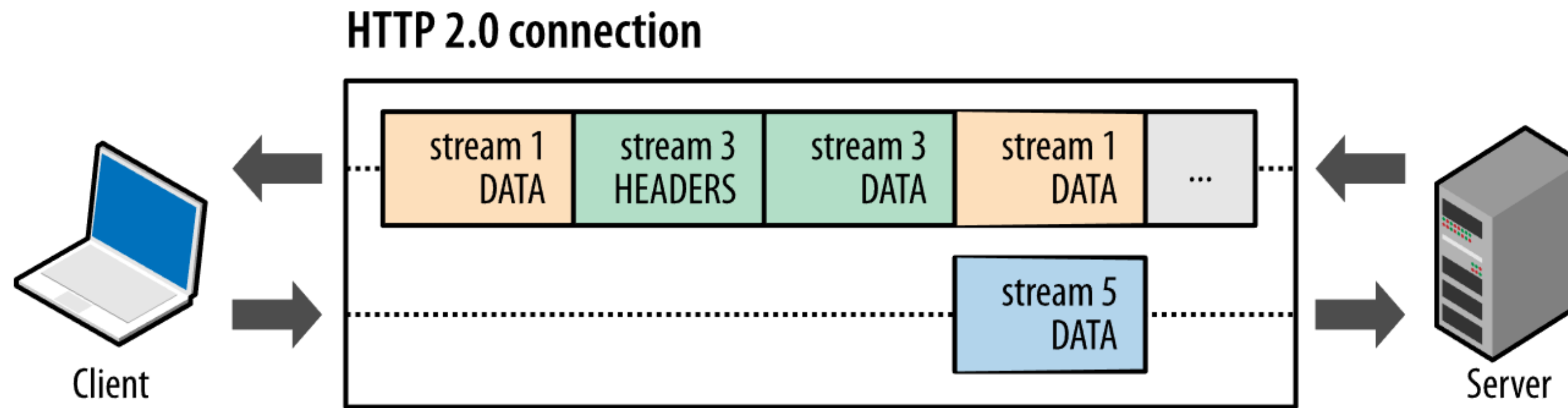
- Multiple TCP connections for each object
- Latency and overhead for opening connections to multiple queries

HTTP/1.1



- Single persistent connection is reused for successive requests
 - **HTTP pipelining** : Introduces multiple, pipelined GETs over a single TCP connection
- Server responds in-order (FCFS: first-come-first-serve scheduling) to GET requests
- With FCFS, small object may have to wait for transmission (**Head-Of-Line (HOL) blocking**) behind large objects
- Loss recovery (**retransmitting lost TCP segments**) stalls object transmission

HTTP/2.0



- Objective: decrease delay in multi-object HTTP requests
- Increased flexibility at server in sending objects to client [RFC 7540, 2015]
 - Methods, status codes, most header fields unchanged from HTTP/1.1
 - Transmission order of request objects based on client-specified object priority (not necessarily FCFS)
 - Push unrequested objects to client
 - **HTTP multiplexing** : divide objects into frame, schedule frames to mitigate HOL blocking

HTTP/2 to HTTP/3

- HTTP/2 over single TCP connection
 - Recovery from packet loss still stalls all object transmissions
 - As in HTTP/1.1, browsers have incentive to open multiple parallel TCP connections to reduce stalling, increase overall throughput
 - No security over vanilla TCP connection
- HTTP/3
 - Adds security, per object error- and congestion- control (more pipelining) over UDP
 - More on HTTP/3 in transport layer

Questions?